

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

Кафедра дискретной математики и информационных технологий

**РАЗРАБОТКА ПРИЛОЖЕНИЯ ДЛЯ АНАЛИЗА
ЭКОНОМИЧЕСКИХ НОВОСТЕЙ**

БАКАЛАВРСКАЯ РАБОТА

Студента 4 курса 421 группы
направления 09.03.01 — Информатика и вычислительная техника
факультета КНиИТ
Тарана Евгения Вячеславовича

Научный руководитель
доцент, к.э.н.

Г. Ю. Чернышова

Заведующий кафедрой
доцент, к. ф.-м. н.

Л. Б. Тяпаев

Саратов 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Анализ методов парсинга	5
1.1 Обзор существующий методов парсинга	5
1.2 Парсинг с использованием обращений к API	7
2 Подходы к анализу тональности текстов	9
2.1 Классические алгоритмы	9
2.2 Словарный подход	14
2.3 Метрики оценки качества моделей	15
3 Реализация приложения для сбора и анализа новостных сообщений ..	17
3.1 Реализация сбора данных	17
3.2 Реализация оценка тональности для новостных текстов.....	28
4 Апробация приложения	37
4.1 Сбор новостей по экономической тематике в региональных СМИ	37
4.2 Пример применения приложения для анализа экономических новостей регионов ПФО.....	39
ЗАКЛЮЧЕНИЕ	42
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	44
Приложение А Программный код реализации графического интерфей- са приложения	46
Приложение Б Программный код реализации парсера	68
Приложение В Программный код реализации анализа тональности.....	75

ВВЕДЕНИЕ

В условиях цифровой трансформации общества объемы неструктурированных текстовых данных, генерируемых в социальных сетях, новостных ресурсах и блогах, растут экспоненциально. Ручной анализ таких данных становится невозможным из-за их масштаба и разнообразия, что делает технологии автоматизированного парсинга и анализа тональности критически важными. Парсинг позволяет извлекать структурированную информацию из источников, таких как социальная сеть ВКонтакте, а анализ тональности даёт возможность интерпретировать эмоциональную окраску текстов, выявляя общественные настроения, предпочтения пользователей или реакцию на события.

Особую актуальность эта тема приобретает для русскоязычного контента, где наблюдается дефицит качественных обучающих выборок для анализа тональности, особенно в узких областях, таких как экономика или политика. Интеграция парсинга с методами машинного обучения и словарными подходами позволяет не только собирать релевантные данные, но и создавать адаптивные инструменты для их интерпретации, что востребовано в маркетинге, управлении репутацией, социологических исследованиях и прогнозировании трендов.

Целью дипломной работы является разработка приложения, объединяющего парсинг текстовых данных из социальной сети ВКонтакте с последующим анализом их тональности с использованием словарных методов.

Для достижения цели поставлены следующие задачи:

- анализ и классификация методов парсинга социальных сетей, выбор оптимального подхода для работы с ВКонтакте;
- разработка модуля парсинга, формирующего текстовые выборки на основе пользовательских запросов;
- разработка модуля анализа тональности текстов с применением словарных методов;
- апробация приложения с целью анализа сформированных данных о региональных экономических новостях и интерпретация результатов.

Объектом исследования являются методы сбора и анализа текстовых данных. Предметом исследования является применение методов анализа тональности новостных сообщений. Практическая и теоретическая значимость

работы заключается в разработке полнофункционального приложения для сбора и анализа тональности новостных сообщений для различных периодов и различных объектов.

Данная работа составлена из четырех разделов. В первом разделе обобщаются подходы к существующим методам парсинга и обосновывается выбор использования в приложении парсинга социальной сети в качестве способа сбора новостных сообщений. Во втором разделе рассматриваются различные методы анализа тональности и поясняется выбор словарей тональности для использования в разрабатываемом приложении. Третий раздел посвящен разработке модуля парсинга социальной сети ВКонтакте и модуля анализа собранных новостных сообщений. В четвертом разделе производится апробация приложения в прикладной задаче анализа динамики тональности новостных сообщений об экономике регионов Приволжского федерального округа.

В бакалаврской работе содержится 1 таблица, 13 рисунков и 20 источников.

1 Анализ методов парсинга

1.1 Обзор существующий методов парсинга

Парсинг веб-сайтов — это процесс извлечения данных из веб-сайтов. Процесс парсинга веб-сайтов включает в себя отправку запросов на получение веб-страницы и извлечение из нее машиночитаемой информации.

Парсинг веб-сайтов является важным инструментом для автоматизации решения аналитических задач. Парсинг веб-сайтов дает следующие возможности:

- автоматизация процессов сбора данных в необходимом масштабе;
- получение доступа к источникам данных в Интернет, которые могут повысить эффективность бизнеса;
- принятие решений с опорой на полученные данные.

Перечисленные факторы объясняют возрастающий интерес к парсингу веб-сайтов как к одному из эффективных способов сбора данных.

В настоящее время существует несколько подходов к осуществлению веб-скрапинга. Одной из задач исследовательской работы стал обзор наиболее популярных методов парсинга и выбор наиболее продуктивного способа для использования его при создании приложения, осуществляющего сбор текстовых данных.

Существует большое количество способов парсинга сайтов. Под различные платформы, такие как Python, Java, Ruby, PHP, JavaScript, существует множество средств, обеспечивающих реализацию парсинга. Для рассмотрения нескольких инструментов веб-скрапинга был выбран язык программирования Python, так как он широко распространен и содержит в себе множество библиотек, осуществляющих реализацию дальнейшей обработки данных, в частности Text Mining. Наиболее популярными инструментами для реализации веб-скрапинга на Python являются библиотека BeautifulSoup и фреймворк Scrapy.

BeautifulSoup — это библиотека для языка Python, которая используется для парсинга HTML и XML документов. Она позволяет извлекать данные из HTML-страницы, такие как заголовки, текст, ссылки, таблицы и многое другое [1].

Основная задача библиотеки BeautifulSoup — это извлечение информации из HTML-кода в структурированном виде. При парсинге докумен-

та BeautifulSoup превращает его в древовидную структуру, где каждый тег представлен в виде узла дерева. После этого можно использовать различные методы и свойства объектов, чтобы получить нужную информацию.

Одна из главных причин популярности использования BeautifulSoup — это то, что библиотека может работать с плохо оформленным HTML-кодом. Даже если страница содержит ошибки в разметке, BeautifulSoup все равно сможет правильно извлечь нужные данные.

BeautifulSoup также поддерживает использование регулярных выражений для поиска элементов на странице и обработки текста. Это делает библиотеку очень гибкой и мощной для извлечения данных из веб-страниц [2].

Scrapy — это фреймворк для языка Python, который используется для создания пауков (spiders) для автоматизированного сбора данных. Scrapy позволяет легко извлекать данные из веб-страниц и сохранять их в различных форматах, таких как CSV, JSON или базы данных [3].

Основным преимуществом Scrapy является его возможность парсить веб-страницы с высокой скоростью и в автоматическом режиме. Фреймворк обладает мощными инструментами для обработки HTML и XML-документов, такими как CSS-селекторы, XPath и регулярные выражения.

Для создания паука в Scrapy необходимо определить несколько ключевых элементов, таких как URL-адреса, которые нужно обойти, и правила для извлечения данных. Затем можно определить методы для обработки данных, например, для очистки и преобразования данных [4].

Scrapy также обладает мощными инструментами обработки больших объемов данных. Он может использоваться для извлечения данных из нескольких сайтов одновременно и параллельной обработки данных.

Существуют способы парсинга, которые позволяют работать без написания кода программы. Один из таких способов осуществляет инструмент Selenium. Selenium — это инструмент для автоматизации действий веб-браузера, который позволяет программно взаимодействовать с веб-страницами, выполнять действия пользователя и получать данные со страницы. Он широко используется для тестирования веб-приложений, автоматизации задач веб-скрапинга и для создания ботов для автоматической работы с веб-сайтами. Также Selenium позволяет программистам имитировать действия пользователя в браузере, такие как нажатие кнопок, заполнение форм и переход по

ссылкам, что может помочь в сборе данных с сайтов, которые желают помешать осуществлению парсинга.

Selenium предоставляет различные инструменты и библиотеки для автоматизации веб-браузера, такие как Selenium WebDriver и Selenium IDE. Selenium WebDriver — это основной инструмент Selenium, который позволяет программно управлять браузером, взаимодействовать с элементами страницы, выполнять действия пользователя и получать данные со страницы. WebDriver поддерживает различные браузеры, такие как Google Chrome, Mozilla Firefox, Microsoft Edge и другие. Selenium Web Driver можно использовать с различными языками программирования, такими как Java, Python, Ruby и другими.

Selenium IDE — это расширение для браузера Firefox, которое позволяет записывать действия пользователя на веб-странице, такие как нажатия кнопок и ввод текста, и проигрывать их снова в тестах. Selenium IDE позволяет быстро создавать автоматические тесты без необходимости написания кода. Однако, он не предоставляет возможности программирования и не поддерживает автоматизацию сложных сценариев.

1.2 Парсинг с использованием обращений к API

Парсинг с использованием различных инструментов языков программирования и скрапинг при помощи средств автоматизации браузера могут быть достаточно эффективными методами сбора информации, эти способы предназначены для сбора информации с практически любых источников. Но существует инструмент парсинга данных, который применим лишь к некоторым информационным ресурсам, создатели которых по собственному желанию предоставляют наиболее простой и наиболее быстрый способ получения большого количества информации с интернет-ресурса. Этот инструмент парсинга называется Application Programming Interface(API).

Application Programming Interface позволяет разработчикам программного обеспечения интегрировать различные функции и сервисы в свои приложения, используя предоставленные интерфейсы. Это может быть функциональность, связанная с облачными вычислениями, базами данных, социальными сетями, платежными системами, геоданными, и т.д. При этом разработчики не должны беспокоиться о том, как работает эта функциональность, но могут использовать ее с помощью предоставленных интерфейсов.

API позволяет упростить разработку программного обеспечения, поскольку разработчики могут использовать уже существующую функциональность, вместо того чтобы писать программный код приложения с нуля.

Одним из главных преимуществ использования API для парсинга данных является его легальность. Если владелец сайта предоставляет API для доступа к своим данным, то использование этого API является законным и не нарушает авторских прав. Это позволяет избежать возможных юридических проблем, связанных с парсингом данных без разрешения владельца сайта [5].

Кроме того, работа с API может быть более эффективной и удобной, чем использование инструментов, таких как BeautifulSoup и Scrapy. API обычно предоставляют структурированные данные в формате JSON или XML, что упрощает их обработку и анализ. API также обычно позволяют получить только нужные данные и избежать извлечения ненужной информации, что ускоряет процесс парсинга и экономит ресурсы.

В то же время, работа через API может обеспечивать более надежную и стабильную работу, так как API обычно являются официальным способом доступа к данным и могут предоставлять гарантии на стабильность и безопасность работы.

Так как именно парсинг с использованием API является самым быстрым и удобным инструментом для извлечения данных, было принято решение разрабатывать приложение, которое будет реализовывать именно этот метод веб-скрапинга. Поскольку целью работы является разработка инструментального средства для формирования текстовой выборки появилась задача поиска такого информационного ресурса, имеющего собственный API, который содержит в себе большое количество различных текстовых данных.

Источником, содержащим в себе большое количество различных текстовых данных, может являться социальная сеть. В силу того, что наиболее популярной русскоязычной социальной сетью является ВКонтакте, многие новостные ресурсы региональных официальных ведомств и СМИ активно ведут новостные сообщества на платформе этого ресурса. Социальная сеть ВКонтакте имеет свой собственный открытый и документируемый API, а также позволяет работать с широким охватом новостных источников, что и стало определяющим фактором в решении обращаться и работать именно с API ВКонтакте.

2 Подходы к анализу тональности текстов

2.1 Классические алгоритмы

Анализ тональности текстов (Sentiment Analysis) — это процесс автоматического определения эмоциональной окраски текстовых данных. Цель анализа тональности заключается в определении, положительное ли, отрицательное или нейтральное отношение выражено в тексте. Анализ тональности применяется в различных областях, включая маркетинг, социологию и управление репутацией.

Для эффективной обработки и понимания эмоционального содержания текста в анализе тональности обычно применяют последовательно несколько этапов.

Прежде всего проводится предварительная обработка текста, которая традиционно представляет собой:

- удаление из текста лишних символов и пунктуации
- удаление стоп-слов (часто встречающихся слов, не несущих смысловой нагрузки)
- удаление других символов, которые не влияют на его тональность
- разбиение текста на отдельные слова или фразы (токены) для дальнейшего анализа

Следующим шагом является преобразование текста в числовой формат (например, TF-IDF веса), чтобы его можно было использовать в алгоритмах машинного обучения.

На следующем этапе происходит применение алгоритмов классификации и регрессии для определения тональности текста на основе извлеченных признаков.

На последнем шаге проводится анализ результатов, заключающийся в интерпретации предсказанных значений тональности и их соответствии реальным эмоциональным оттенкам текста.

В области компьютерной лингвистики анализ тональности традиционно реализуется через два принципиально различных подхода: классические алгоритмы машинного обучения и методы глубокого обучения. Каждый из этих подходов обладает характерными преимуществами и ограничениями, определяющими область их эффективного применения.

Классические алгоритмы, включая методы на основе логистической ре-

грессии, метода опорных векторов (SVM) и наивного байесовского классификатора, опираются на статистический анализ текстовых признаков, чаще всего представленных в виде векторизованных моделей "мешка слов" (Bag-of-Words) или их взвешенных аналогов (TF-IDF). Данные подходы демонстрируют относительно высокую эффективность при работе с короткими текстами фиксированной структуры, однако имеют принципиальные ограничения, связанные с игнорированием семантических и синтаксических зависимостей между словами. Кроме того, их эффективность существенно снижается при обработке текстов с высокой степенью лексического разнообразия или содержащих сложные лингвистические конструкции (ирония, сарказм, метафорические выражения). Важным недостатком является также необходимость тщательного инжиниринга признаков и ручной настройки параметров модели для каждого конкретного домена.

Рассмотрим примеры классических алгоритмов. Дерево решений (Decision Tree) представляет собой алгоритм машинного обучения, применяемый как для задач классификации, так и для регрессионного анализа. В контексте анализа эмоциональной окраски текстов данный метод позволяет относить текстовые данные к категориям позитивной, негативной или нейтральной тональности на основе выявленных закономерностей.

Деревья решений находят широкое применение в задачах анализа тональности текстов благодаря своей прозрачности и относительной простоте интерпретации. Этот метод классификации особенно полезен при работе с текстовыми данными, где важно понимать, какие именно слова или сочетания слов влияют на итоговую эмоциональную оценку. Основной принцип работы деревьев решений заключается в последовательном разделении текстовых данных на группы по наиболее значимым признакам. В контексте анализа тональности такими признаками обычно выступают отдельные слова или их сочетания. Алгоритм начинает с корневого узла, где выбирается наиболее информативный признак, и затем рекурсивно разделяет данные на подмножества, пока не достигается заданный критерий останова [6].

Применительно к анализу тональности, построение дерева решений начинается с преобразования текста в числовые признаки, такие как частоты слов или наличие определенных лексических маркеров. Затем выбирается критерий разделения (чаще всего используется индекс Джини или энтро-

пия), который определяет, какие слова наиболее четко разделяют тексты по эмоциональной окраске.

Важной особенностью использования деревьев решений для анализа тональности является необходимость тщательной предварительной обработки текста. Стемминг и лемматизация помогают сократить размерность признакового пространства, а балансировка классов предотвращает перекося в сторону доминирующей эмоциональной категории. На практике деревья решений часто комбинируют с ансамблевыми методами, такими как случайный лес, что позволяет повысить точность классификации при сохранении интерпретируемости результатов [7].

Общий алгоритм построения дерева принятия решений выглядит следующим образом:

1. Выбирается очередной атрибут, и помещается в корень.
2. Для всех его значений i :
 - остаются из тестовых примеров только те, у которых значение атрибута равно i ;
 - рекурсивно строится дерево в этом потомке.

Пример построения дерева решений представлен на рисунке 1.

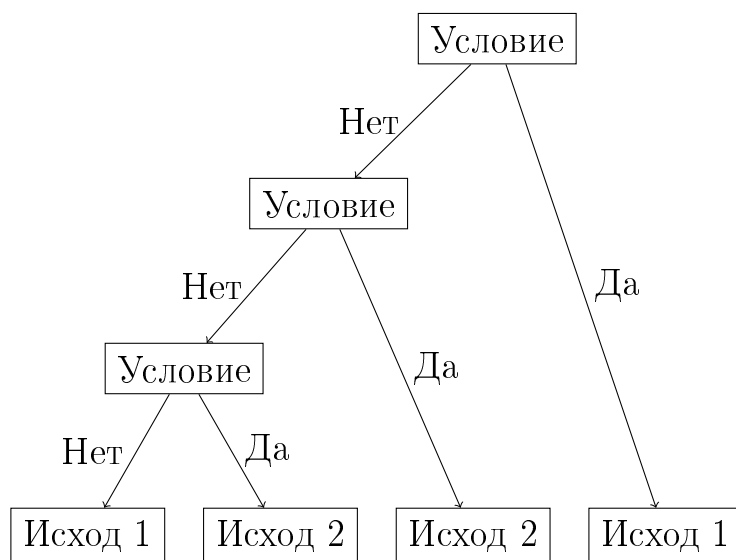


Рисунок 1 – Пример построения дерева решений

Одним из ключевых недостатков деревьев решений является их предрасположенность к переобучению, которое особенно проявляется при работе с объемными и сложными наборами данных. Для преодоления этого ограничения были разработаны модификации алгоритма, среди которых наиболее

эффективной оказалась методика случайного леса.

Случайный лес (Random Forest) представляет собой ансамблевый алгоритм машинного обучения, который использует комбинацию множества деревьев решений для повышения точности прогнозирования. В основе этого подхода лежит техника бутстрэп-агрегирования (bagging), позволяющая снизить эффект переобучения и увеличить устойчивость модели [8].

Характерные особенности алгоритма случайного леса включают:

- использование случайного подмножества признаков при построении каждого дерева, что снижает корреляцию между отдельными деревьями и минимизирует риск переобучения;
- формирование обучающей выборки для каждого дерева методом случайного выбора с возвращением (bootstrap sampling) из исходных данных;
- агрегацию результатов всех деревьев через усреднение (для задач регрессии) или выбор наиболее частотного класса (для задач классификации) для получения итогового прогноза.

Алгоритм случайного леса демонстрирует высокую устойчивость к переобучению, обладает хорошей обобщающей способностью и показывает высокую эффективность при работе с разнородными данными, включая текстовую информацию в задачах определения эмоциональной окраски.

Метод k-ближайших соседей (k-Nearest Neighbours, k-NN) представляет собой простой, но действенный алгоритм машинного обучения, применимый для анализа тональности текстов. Основная идея метода заключается в предположении, что схожие по характеристикам объекты обычно принадлежат к одним и тем же классам.

Применение алгоритма k-NN для анализа эмоциональной окраски текстов включает следующие этапы [9]:

1. векторизация текстовых данных с использованием методов Bag of Words или TF-IDF, после чего каждому тексту присваивается соответствующая метка класса (позитивный, негативный или нейтральный);
2. выбор метрики расстояния между векторными представлениями текстов. Для текстовой информации часто применяется косинусная мера, учитывающая угловое расстояние между векторами;
3. определение оптимального количества соседей (параметр k), участву-

ющих в классификации. Этот гиперпараметр существенно влияет на качество работы модели;

4. процедура классификации новых текстовых данных:

- вычисление расстояний между новым текстом и всеми образцами обучающей выборки;
- отбор k ближайших текстов с минимальными расстояниями;
- определение класса нового текста на основе преобладающего класса среди выбранных соседей (методом голосования).

Применительно к анализу тональности текстов метод k -ближайших соседей может показывать высокую эффективность в случаях, когда данные обладают сложной структурой и между текстами существуют выраженные тематические связи. Однако данный подход имеет ряд существенных ограничений, среди которых следует отметить высокую вычислительную сложность при работе с большими объемами данных и признаковыми пространствами высокой размерности.

Одной из ключевых проблем при использовании k -NN для анализа тональности становится необходимость корректного выбора метрики близости и оптимального значения параметра k , которые существенно зависят от специфических характеристик обрабатываемых данных. Важно также учитывать, что данный метод не принимает во внимание потенциальные взаимосвязи между признаками и может демонстрировать повышенную чувствительность к наличию шумовых данных в выборке.

Для корректной оценки эффективности алгоритмов машинного обучения требуется предварительное разделение исходного набора данных на обучающую и тестовую подвыборки. Обучающая часть данных служит для построения модели, тогда как тестовая предназначена для объективной оценки её качества. В ряде случаев дополнительно выделяют валидационную выборку, которая помогает в тонкой настройке параметров модели.

При разделении данных рекомендуется учитывать принцип стратификации, который сохраняет исходное распределение классов в каждой из подвыборок. Такой подход предотвращает потенциальное смещение модели в сторону доминирующего класса и обеспечивает репрезентативность выборок.

Методы глубокого обучения, в частности рекуррентные (RNN, LSTM, GRU) и сверточные (CNN) нейронные сети, а также современные трансфор-

мерные архитектуры (BERT, RoBERTa), позволяют преодолеть многие из указанных ограничений за счет автоматического извлечения лингвистических признаков и учета контекстной семантики. Наиболее продвинутые модели, основанные на механизме внимания (attention), демонстрируют state-of-the-art результаты на стандартных тестовых наборах данных. Однако их применение сопряжено с рядом существенных проблем: необходимостью наличия больших размеченных корпусов текстов, высокими вычислительными затратами на обучение, а также принципиальной сложностью интерпретации принимаемых решений (проблема "черного ящика"). Кроме того, эффективное использование таких моделей требует специализированного аппаратного обеспечения (GPU/TPU) и глубоких знаний в области настройки гиперпараметров, что существенно ограничивает область их практического применения, особенно в условиях ограниченных ресурсов.

Далее описанные методы k-ближайших соседей и случайного леса будут реализованы для проведения вычислительного эксперимента оценки точности алгоритмов.

2.2 Словарный подход

Словарные методы анализа тональности основываются на использовании предопределенных словарей, которые содержат слова и фразы с заранее определенной эмоциональной окраской (положительной, отрицательной или нейтральной). Преимущества словарных методов включают:

- простоту реализации и интерпретации результатов;
- отсутствие необходимости в обучающих данных;
- быстроту работы по сравнению с методами машинного обучения
- прозрачность процесса анализа (легко понять, какие слова повлияли на результат);
- возможность адаптации под конкретную предметную область путем дополнения словаря.

Словарные подходы к анализу тональности обладают рядом существенных ограничений при работе с экономическими новостными текстами. Основные проблемы включают: ограниченный охват специализированной экономической лексики, отсутствие учета контекста употребления слов (что критично для многозначных терминов), неспособность корректно обрабатывать идиоматические выражения и устойчивые словосочетания. Дополнительные

сложности создает динамичность новостного языка, требующая регулярного обновления словарных ресурсов. Большинство существующих словарей созданы для общего использования и не учитывают специфику экономической тематики, что существенно снижает качество анализа в данной предметной области [10].

Учитывая указанные особенности, для реализации анализа тональности в рамках бакалаврской работы был выбран словарный подход как методологически наиболее соответствующий поставленным задачам. Данный выбор обусловлен следующими объективными факторами: отсутствием необходимости в размеченных обучающих данных, низкими вычислительными требованиями, высокой степенью интерпретируемости результатов. Хотя словарные методы теоретически уступают в точности современным нейросетевым подходам, их преимущества в условиях ограниченных временных и вычислительных ресурсов, типичных для учебных исследований, делают их оптимальным выбором. Кроме того, модульная структура словарных подходов позволяет в дальнейшем легко комбинировать их с другими методами, создавая гибридные системы анализа тональности.

Таким образом, проведенный анализ стал причиной выбора словарных методов для решения поставленной задачи. Их применение позволяет получить научно значимые результаты при соблюдении требований к методологической строгости и в рамках доступных ресурсов, что особенно важно для учебно-исследовательской работы уровня бакалавриата.

2.3 Метрики оценки качества моделей

При оценке моделей анализа тональности текстов (Sentiment Analysis), как и в других задачах классификации, применяют различные метрики качества. Выбор конкретных метрик определяется особенностями задачи и целями исследования. Рассмотрим наиболее распространённые метрики, которые будут использованы в работе [11].

TP (True Positives) – количество корректно идентифицированных положительных высказываний. TN (True Negatives) – количество верно распознанных отрицательных высказываний. FP (False Positives) – количество ошибочно классифицированных как положительные примеры. FN (False Negatives) – количество ошибочно классифицированных как отрицательные примеры.

Общая точность (Accuracy) показывает долю верно классифицированных примеров среди всех предсказаний:

$$\text{Accuracy} = \frac{TN + TP}{TN + TP + FN + FP}$$

В задачах с несбалансированными классами эта метрика может давать искажённую оценку качества.

Точность (Precision) характеризует долю истинно положительных случаев среди всех предсказанных положительных:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Точность особенно важна в задачах, где критичны ложные срабатывания.

Полнота (Recall) показывает долю верно предсказанных положительных примеров среди всех фактически положительных:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Полнота имеет значение, когда важно минимизировать пропуск положительных случаев.

F1-мера представляет собой гармоническое среднее между точностью и полнотой, особенно полезное для несбалансированных данных:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Матрица ошибок (Confusion Matrix) является важным инструментом оценки, который наглядно представляет распределение предсказаний модели по четырём категориям: TP, TN, FP, FN. Это позволяет комплексно оценить производительность классификатора.

Представленные метрики будут использованы для оценки качества моделей, обученных на сгенерированной выборке, что позволит косвенно оценить и саму выборку.

3 Реализация приложения для сбора и анализа новостных сообщений

3.1 Реализация сбора данных

Для формирования текстовых выборок с новостными материалами существует ряд методов получения данных, включая парсинг веб-страниц, а также использование API (англ. Application Programming Interface) социальных платформ.

Так как именно парсинг с использованием API является самым быстрым и удобным инструментом для извлечения данных, было принято решение разрабатывать приложение, которое будет реализовывать именно этот метод веб-скрапинга посредством работы с API социальной сети Вконтакте.

У Вконтакте есть собственный API (VKontakte API), созданный для создания Standalone-приложений и приложений внутри самой соцсети. С помощью VK API можно получать доступ к данным пользователя, сообщества или группы, а также выполнять различные действия, такие как публикация постов, отправка сообщений, получение информации о друзьях и т.д. [12].

VK API работает на основе протокола HTTP и использует формат передачи данных JSON. Для доступа к API необходимо зарегистрировать свое приложение на странице для разработчиков VK, после чего получить ключи доступа, которые необходимы для отправки запросов к API.

Методы работы с VK API по сути реализуют ряд операций с базами данных. Эти методы разделены на секции в зависимости от того, в какой сфере и к чему метод применяются. Так, например, для работы с сообществами используются методы секции Groups, а для работы с сообщениями — методы секции Messages.

Чтобы обратиться к методу VK API необходимо выполнить get-запрос по шаблону вида:

```
https://api.vk.com/method/<название метода>?<параметры>
```

Для работы с методами API необходимо передавать два обязательных параметра и несколько необязательных. Обязательные параметры одинаковы для всех методов: `access_token` (ключ доступа) и `v` (версия API). Необязательные параметры индивидуальны для каждого метода и подробно описаны

в документации. Например, набор параметров для метода `search` из секции `Groups` показан на рисунке 2.

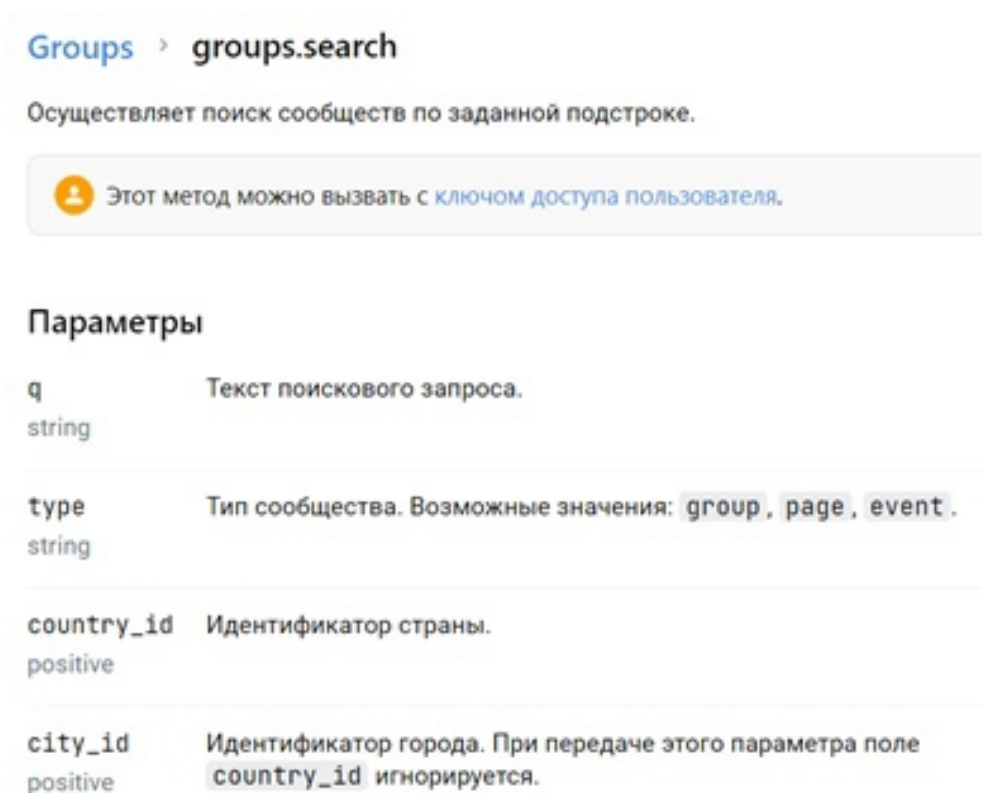


Рисунок 2 – Параметры метода `Search_Groups`

Для доступа к API требуются ключи двух типов: сервисные (для работы от имени приложения) и пользовательские (для работы от имени пользователя Вконтакте). При разработке парсера данных потребовалось использование обоих типов ключей - сервисные для базовых операций, пользовательские для работы с персональными данными.

После получения ключа доступа можно выполнить первый HTTP-запрос к VK API. В качестве примера рассмотрим метод `get` из секции `Wall`, который позволяет получать записи со стены пользователя или сообщества. Сформированный по указанному ранее шаблону URL-адрес запроса можно ввести в адресную строку браузера. В ответ система вернет страницу с данными в формате JSON.

Дальнейшая разработка парсера текстов будет связана с обработкой данных в формате JSON. Для удобства работы полученную информацию потребуется преобразовать и структурировать.

В рамках работы над бакалаврской работой был разработан модуль приложения, способный выполнять следующие функции: ввод ключевых слов,

задание временного периода, указание новостных источников, сбор данных по ключевым словам, отображение количества найденных текстов и сохранение результатов в файл формата `xlsx`.

Парсер и другие модули приложения были реализованы на языке Python 3.9 с использованием интегрированной среды разработки PyCharm Professional Edition. Python представляет собой интерпретируемый язык программирования высокого уровня, обладающий объектно-ориентированной парадигмой и динамической семантикой. Его ключевыми преимуществами являются высокоуровневые структуры данных, динамическая типизация и привязка, что делает его особенно эффективным для быстрой разработки приложений. Язык отличается интуитивно понятным синтаксисом, который способствует улучшению читаемости кода и снижению затрат на его поддержку. Кроме того, Python поддерживает модули и пакеты, что обеспечивает модульность программ и возможность повторного использования кода.

Модуль `requests` в Python предназначен для упрощенной работы с HTTP-запросами. В нем реализованы различные методы для взаимодействия с веб-ресурсами. При создании парсера использовался наиболее распространенный метод `get`. Метод `requests.get` принимает в качестве параметра полный URL-запроса или его составные части. Поскольку метод возвращает значение, после выполнения запроса он предоставляет объект класса `response` для дальнейшей обработки.

Применяя метод `requests.get` совместно с функциями VK API, были разработаны методы сбора публикаций из сообществ Вконтакте и соответствующих комментариев. Параметрами для этих методов выступают: название паблика, временной диапазон публикаций и ключевые слова.

Перед использованием ключевых слов для фильтрации публикаций по тематике требуется их предварительная обработка. Она включает стемминг и формирование регулярного выражения для расширения охвата релевантных публикаций. Приведем пример из кода, выполняющего такую подготовку.

```
# Инициализация анализатора pymorphy3
morph = pymorphy3.MorphAnalyzer()

regular_expression = ''

keywords = words.lower().split(',')
```

```

if len(keywords[0].split()) > 1:
    for keyword in keywords[0].split():
        # Используем pymorphy3 для нормализации слова
        normalized_word = morph.parse(keyword)[0].normal_form
        regular_expression += f'{normalized_word}.*'
    regular_expression += '|'

    for keyword in reversed(keywords[0].split()):
        normalized_word = morph.parse(keyword)[0].normal_form
        regular_expression += f'{normalized_word}.*'
else:
    normalized_word = morph.parse(keywords[0])[0].normal_form
    regular_expression += normalized_word

if len(keywords) > 1:
    for i in range(1, len(keywords)):
        regular_expression += '|'

        for keyword in keywords[i].split():
            normalized_word = morph.parse(keyword)[0].normal_form
            regular_expression += f'{normalized_word}.*'

        regular_expression += '|'

        for keyword in keywords[i].split():
            normalized_word = morph.parse(keyword)[0].normal_form
            regular_expression += f'{normalized_word}.*'

```

Для сбора публикаций за указанный период используется метод VK API `Wall.Get`, позволяющий получать до 100 записей со стены открытого сообщества за один запрос. Чтобы увеличить количество собираемых публикаций, в код была введена переменная `offset`, контролирующая смещение относительно начальной записи при выборке.

Для сбора публикаций за заданный временной интервал используется метод `Wall.Get` из VK API. Данный метод позволяет получить до 100 записей со стены открытого сообщества при каждом обращении. Чтобы обеспечить сбор большего количества постов, в реализации был применен параметр `offset`, который определяет смещение относительно начальной записи при выборке данных.

При работе с API возникла проблема чрезмерно частых запросов к серверу, что приводило к сбоям в работе приложения. Решением стало использование специального метода `execute` из VK API. Этот метод выполняет переданный ему код на VKScript, позволяя вызывать другие API-методы с возможностью сохранения и обработки промежуточных результатов.

VK Script — это специализированный язык программирования, разработанный внутри компании VK для автоматизации задач в социальной сети. Являясь диалектом JavaScript, он включает специфичные функции и методы, адаптированные для работы с платформой VKontakte.

Основные преимущества VK Script включают простоту использования и наличие готовых функций для автоматизации типовых операций: отправки сообщений, получения списка контактов, взаимодействия с записями и других задач внутри социальной сети.

Использование метода `execute` и языка VK Script для запросов к VK API дает ключевое преимущество - возможность выполнять значительно больше запросов в минуту в рамках ограничений, установленных разработчиками Вконтакте. Это решение существенно уменьшило время выполнения приложения. Приведем пример реализации обращения к методу `Wall.Get` с использованием метода `execute`.

```
while offset < posts_count:
    # Формируем код для VK API запроса с текущим offset
    code1 = (
        """return API.wall.get({"domain": "" + f'{{domain}}' +
        """, "offset": "" + str(offset) +
        """, "count": 100, "extended": 0, "v": "5.131"});"""
    )

    # Отправляем запрос к VK API с использованием метода execute
    response1 = requests.post(
        "https://api.vk.com/method/execute",
        data={
            "code": code1,
            "access_token": token,
            "v": version
        }
    )
```

```

# Получаем список постов из ответа
data1 = response1.json()[ 'response '][ 'items ' ]

# Прерываем цикл, если достигли постов старше dt1
if data1[50][ 'date ' ] < dt1:
    break

# Фильтруем посты по регулярному выражению и дате
for post in data1:
    post_text = post[ 'text ' ].lower()
    post_date = post[ 'date ' ]

    if (re.search(rf"{regular_expression}", post_text) is not
        None and
        dt1 < post_date < dt2):
        searched_posts.append(post)

# Увеличиваем offset для следующей порции постов
offset += 100

# Обновляем время выполнения
end = time.time() — start

# Добавляем найденные посты в общий список и очищаем временный
all_searched_posts.extend(searched_posts)
searched_posts.clear()

```

Сбор комментариев к постам через метод Comments.Get реализован аналогично сбору публикаций из сообществ Вконтакте с использованием VK Script. Следует отметить, что при обработке комментариев, требующей в сотни раз больше запросов к серверу, время работы приложения могло бы быть чрезмерно большим без применения метода execute.

После разработки функций сбора комментариев и публикаций из конкретного сообщества возникла необходимость в реализации поиска и обращения к нескольким группам со схожими названиями в рамках одной функции. С реализацией этой функции можно ознакомиться в Приложении А.

Одной из ключевых функций парсера является сохранение собранных данных в файл формата xlsx, который оптимально подходит для последующей обработки текстовых выборок. Для работы с данным форматом был

использован стандартный модуль Python - `openpyxl` [13].

Для разработки графического интерфейса была выбрана библиотека Tkinter и пакет CustomTkinter. Tkinter представляет собой стандартную библиотеку Python для создания графических интерфейсов, основанную на кроссплатформенной библиотеке Tk, изначально созданной для языка Tcl, но затем адаптированной для Python и других языков программирования [14].

Основное преимущество Tkinter заключается в простоте использования и быстрой интеграции с Python, позволяя создавать базовые GUI-приложения минимальным количеством кода. Библиотека включает набор стандартных виджетов: кнопки, текстовые поля, метки и другие элементы управления. Дополнительными достоинствами Tkinter являются кроссплатформенность, активное сообщество разработчиков, подробная документация и возможность создания пользовательских виджетов.

CustomTkinter расширяет функциональность стандартной библиотеки, предлагая дополнительные виджеты (календари, графики, таблицы) и возможности кастомизации интерфейса, включая предустановленные стили оформления [15].

После конфигурации основного окна приложения потребовалось создать и настроить элементы интерфейса: текстовые метки, индикатор прогресса, кнопки управления, поля выбора и ввода данных. Стилистическое оформление элементов осталось практически без изменений, были использованы стандартные стили, предоставляемые пакетом CustomTkinter.

Для интеграции графического интерфейса с парсером разработаны две функции: `get_and_parse_posts` и `get_and_parse_comments`, которые вызывают соответствующие методы парсера `main_parse_posts` и `main_parse_comments`. Реализация методов всех методов парсинга приведена в Приложении Б. Приведем реализацию функции `get_and_parse_posts` (функция `get_and_parse_comments` имеет аналогичную структуру).

```
def get_and_parse_posts(one_theme):
    """Получает и парсит посты по заданной теме с учётом параметров
        фильтрации. """
    # Получаем параметры из интерфейса
    filename = entry_folder_parse.get()
    date_from = entry_from.get()
    date_to = entry_to.get()
```

```

city = one_theme
group_count = entry_group_count.get()
min_word_count = entry_min_words.get()
keywords = entry_keywords.get()

# Проверяем, что все обязательные поля заполнены
if not all([city, date_from, date_to, filename, group_count,
            min_word_count]):
    status_label.configure(text="Данных недостаточно")
    status_label.pack()
    return []
try:
    # Преобразуем даты из строк в datetime объекты
    dt_from = datetime.strptime(date_from, '%d.%m.%Y')
    dt_to = datetime.strptime(date_to, '%d.%m.%Y')

    # Обновляем статус и начинаем парсинг
    status_label.configure(
        text=f"Парсинг в сообществах близких к теме {one_theme}
              ..."
    )
    # Основная функция парсинга
    posts_by_city = parsar.main_parse_posts(
        city=city,
        date_from=dt_from.timestamp(),
        date_to=dt_to.timestamp(),
        group_count=group_count,
        filename=filename,
        min_word_count=min_word_count,
        keywords=keywords,
        token=entry_token.get()
    )

    # Показываем результат
    status_label.configure(text=f"Отобрано {len(posts_by_city)}
                                постов")
    time.sleep(2) # Даём время прочитать статус
    return posts_by_city

except ValueError as e:
    status_label.configure(text=f"Ошибка формата даты: {e}")

```

```

        return []
    except Exception as e:
        status_label.configure(text=f"Ошибка при парсинге: {e}")
        return []

```

Основная функция интерфейса UI активируется при нажатии кнопки "Запуск". Она выполняет сбор данных из всех полей ввода, преобразует даты в требуемый формат, блокирует элементы управления на время выполнения операции.

Метод `get_and_parse_themes` в зависимости от выбранных пользователем параметров вызывает либо функцию `get_and_parse_comments`, либо `get_and_parse_posts`, либо последовательно обе функции. Эти функции заполняют изначально пустые массивы, содержащие собранные посты и комментарии. Упомянутые функции реализуют сценарий, когда требуется сбор как постов, так и комментариев по заданным тематикам.

После завершения работы в любом из режимов функция под названием `get_and_parse_themes` вызывает соответствующую функцию для отображения результатов работы парсера в интерфейсе приложения.

После реализации основной функции графического интерфейса оставалось только организовать её вызов. Чтобы вызов главной функции интерфейса, содержащей длительные операции парсера, не блокировал GUI до их завершения было принято решение использовать в месте вызова лямбду-функцию, которая при нажатии кнопки запускает функцию под названием `get_and_parse_theme` в отдельном фоновом потоке (демо-не).

Разработанные элементы графического интерфейса и механизмы его взаимодействия с парсером предоставляют пользователю возможность настраивать параметры для классификации текстовых данных и инициировать процесс сбора информации из социальной сети Вконтакте.

Рассмотрим практическое применение разработанного парсера для сбора новостных сообщений. В качестве примера зададим следующие параметры поиска:

- ключевые слова: "быстро развивающиеся производства; климат для бизнеса, степень монополизации рынков, состояние рынка недвижимости, уровень регионального планирования, деловая инфраструктура, объемы общественных и частных инвестиций, состав и объем муниципальных услуг, инфраструктура, транспорт и связь, степень развития

- малого предпринимательства";
- временной период: "01.01.2022-01.01.2023";
- ключевые слова для поиска сообществ: "Саратов, Саратовская область";
- количество анализируемых сообществ по каждой теме: 3;
- режим работы: "Сбор постов";
- минимальная длина текста: 20 слов.

На рисунке 3 представлен интерфейс модуля «Парсинг», отвечающего за сбор данных, с заполненными полями.

The screenshot shows a web application window titled 'ParseVK'. The 'Парсинг' (Parsing) tab is active, with other tabs being 'API Token', 'Статистика' (Statistics), and 'Сводка' (Summary). The interface includes several input sections: 'Укажите период' (Specify period) with 'От' (From) and 'До' (To) date pickers set to 01.01.2022 and 01.01.2023, and a 'Год' (Year) dropdown set to 2022; 'Выборка' (Selection) with input fields for 'Ключевые слова для поиска сообществ' (Keywords for community search), 'Количество сообществ' (Number of communities), 'Мин. количество слов' (Minimum number of words), and 'Ключевые слова' (Keywords); 'Запись в файл' (Save to file) with an 'Имя файла' (File name) input field; and checkboxes for 'Посты' (Posts) and 'Комментарии' (Comments). A progress bar is partially filled, and the text 'Ожидание запуска...' (Waiting for launch...) is displayed above a blue 'Запуск' (Launch) button.

Рисунок 3 – Вкладка с настройками парсинга

Таким образом, функционал вкладки «Парсинг» позволяет пользователю после отработки одного цикла сбора и отбора данных сформировать срез данных. Срез данных — это подмножество данных, полученное из более крупного массива или набора данных путём выборки определённых элементов по заданным критериям. Критериями для формирования срезов в приложении являются ключевые слова, которые обязаны присутствовать в тексте, минимальное количество слов в тексте и временной промежуток, в который новостное сообщение должно быть опубликовано.

Для того, чтобы начать пользоваться функциями сбора пользователю необходимо получить `user_token` (токен пользователя), который можно получить через OAuth-авторизацию. Для его получения при работе с приложением пользователю нужно нажать на кнопку, перенаправляющую пользователя по специальной ссылке:

```
https://oauth.vk.com/authorize?client_id=CLIENT_ID&display=page  
&redirect_uri=https://oauth.vk.com/blank.html&scope=offline  
&response_type=token&v=5.131&state=123456
```

После авторизации пользователь будет перенаправлен на URL вида:

```
https://oauth.vk.com/blank.html#access_token=USER_TOKEN...
```

После этого, ему необходимо извлечь параметр `access_token` из URL — это и есть необходимый `user_token`, который перед началом сбора нужно ввести в соответствующее поле.

Для получения и ввода токена пользователя в приложении была реализована специальная вкладка. Графический интерфейс этой вкладки можно увидеть на рисунке 4.

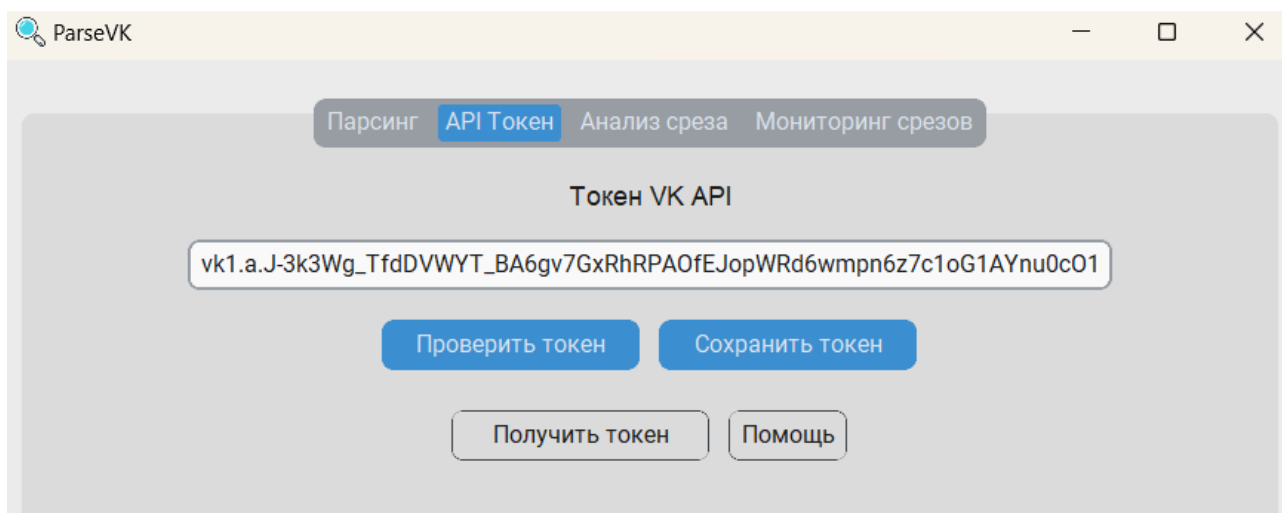


Рисунок 4 – Вкладка для получения и ввода токена

Таким образом, модуль парсинга позволяет пользователю получить необходимый для работы приложения токен пользователя VK API, выбрать параметры поиска источников текстовых сообщений, установить критерии отбора текстов для финальной выборки и сохранить эту выборку в файл по указанному пути.

3.2 Реализация оценка тональности для новостных текстов

Одной из задач исследовательской работы является реализация анализа тональности текстов с применением словарных методов. Для достижения данной задачи был произведен поиск подходящих словарей тональности русских текстов. В качестве универсальных вариантов были выбраны словарь RuSentiLex_2017 и словарь kartaslovsent.

RuSentiLex — это русскоязычный словарь для анализа тональности текстов. В нем слова и словосочетания помечены как позитивные, негативные или нейтральные. Формат словаря следующий: каждая строка содержит само слово, часть речи, его лемму (нормальную форму), тональность (positive/negative/neutral) и дополнительную информацию, например, основана ли оценка на фактах или мнениях. RuSentiLex содержит 9744 негативных, 3585 позитивных, 1394 нейтральных текста [16].

Словарь имеет следующую утвержденную структуру, хранящуюся в файле формата .txt:

1. слово или словосочетание;
2. часть речи или синтаксический тип группы;
3. слово или словосочетание в лемматизированной форме;
4. тональность: позитивная (positive), негативная (negative), нейтральная (neutral) или неопределенная оценка (positive/negative);
5. источник: оценка (opinion), чувство (feeling), факт (fact);

Для корректной работы с данным тональным словарем был написан python-script, конвертирующий его в формат csv, исключая лишние столбца и переводящий тональность из текстовой формы в числовую: positive(1), negative(-1), neutral(0), positive(0).

Приведем код данного скрипта на языке Python.

```
# Функция для преобразования тональности в число
def sentiment_to_num(sentiment):
    sentiment = sentiment.lower().strip()
    if sentiment == 'positive':
        return 1
    elif sentiment == 'negative':
        return -1
    else: # neutral, positive/negative и другие случаи
        return 0
```

```
# Чтение исходного файла и запись в CSV
def convert_txt_to_csv(input_file, output_file):
    with open(input_file, 'r', encoding='utf-8') as infile, \
        open(output_file, 'w', encoding='utf-8', newline='') as outfile:
        writer = csv.writer(outfile)
        writer.writerow(['lemma', 'sentiment']) # Заголовок CSV
        for line in infile:
            parts = line.strip().split(',')
            if len(parts) >= 4:
                lemma = parts[2] # Лемматизированная форма
                sentiment = parts[3] # Тональность
                sentiment_num = sentiment_to_num(sentiment)
                writer.writerow([lemma, sentiment_num])
```

Вторым словарем для добавления в приложения был выбран KartaslovSent. Kartaslov — это проект, связанный с обработкой естественного языка (NLP) для русского языка. В рамках этого проекта существует KartaslovSent, который представляет собой словарь тональностей, похожий на RuSentiLex, но с некоторыми отличиями.

KartaslovSent содержит слова и выражения с их эмоциональной окраской: позитивные, негативные и нейтральные [17]. В отличие от RuSentiLex, KartaslovSent может включать более современную лексику и разметку, а также дополнительные метаданные, такие как сила эмоциональной окраски или контекстные варианты употребления.

Формат данных в KartaslovSent обычно структурирован в виде CSV или JSON, что удобно для загрузки в Python. Например, каждая запись может содержать само слово, его лемму, часть речи, тональность (в виде числа или метки) и, возможно, примеры употребления.

Формирование данного словаря осуществлялось посредством краудсорсинговой разметки, предполагающей ручную аннотацию единиц словаря. В процессе аннотирования участникам исследования предоставлялась возможность классифицировать каждую лексическую единицу по четырём категориям: «позитивная», «негативная», «нейтральная» или «неопределенная».

Результирующий словарный ресурс содержит дополнительный атрибут — агрегированную оценку тональности для каждой лексической единицы. По итогам аннотации был сформирован корпус, насчитывающий 46127 лексем, в котором содержится 6 215 лексем (13,5%) с позитивной маркировкой,

11 863 лексем (25,7%) с негативной маркировкой и 28 049 лексем (60,8%) с нейтральной маркировкой [17].

Полученное распределение демонстрирует характерную для большинства естественных языков асимметрию в представленности эмоционально окрашенной лексики, где нейтральные единицы составляют значительное большинство, что соответствует лингвистическим закономерностям организации лексикона естественного языка[18].

В ходе создания приложения было принято решение использовать бинарную классификацию текстов на позитивные тексты(1) и объединение негативных сообщений с нейтральными(0). Кроме того, в ходе применения словаря был выработан порог для классификации. Если средняя оценка больше 0.2, текст признается позитивным, иначе он считается нейтральным или негативным.

За определение тональности текста отвечает функция под названием `determine_sentiment_score`, принимающая в качестве параметров текст для анализа и идентификатор, определяющий, какой словарь применять для определения тональности. Приведем реализацию данной функции.

```
def determine_sentiment_score(text: str, dict_id: int) -> float:
    """ Определяет тональную оценку текста на основе заданного
        словаря ."""

    # Выбираем нужный словарь
    csv_name = "kartaslovsent.csv" if dict_id == 1 else "rusentilex.
        csv"
    sentiment_csv_path = resource_path(csv_name)
    sentiment_dict = load_sentiment_dict(sentiment_csv_path, dict_id
        )

    # Проверяем что текст не пустой
    if not isinstance(text, str) or not text.strip():
        return 0.0

    words = text.split()
    score = 0
    count = 0

    # Анализируем каждое слово в тексте
    for word in words:
```

```

try:
    # Нормализуем слово и проверяем его в словаре
    normal_form = morph.parse(word.lower())[0].normal_form
    if (normal_form in sentiment_dict and
        normal_form not in russian_stopwords):
        score += sentiment_dict[normal_form]
        count += 1
except Exception:
    continue # Пропускаем слова с ошибками обработки

# Если не найдено подходящих слов, сохраняем текст для анализа
if count == 0:
    df_left.append(text)
    return 0.0

# Рассчитываем среднее и определяем итоговую оценку
average_score = score / count
return 1.0 if average_score >= 0.2 else 0.0

```

Реализация всех функций анализа тональности приведена в Приложении В.

Для того, чтобы убедиться в точности классификации текстов с использованием рассматриваемых словарей было принято решение разметить вручную новостные сообщения об экономике, собранные с помощью модуля парсинга. Была построена специальная выборка, содержащая 422 новостных сообщения, среди которых было признано позитивными 260 и негативными или нейтральными 162.

В целях проверки обоснованности использования словарей тональности в качестве инструмента для анализа тональности новостных сообщений на экономическую тематику, было принято решение организовать сравнение словарных методов с рассмотренными в разделе 2.1 классическими алгоритмами классификации на специальной тестовой выборке. Для построения обучающей выборки, содержащей новостные сообщения на экономическую тематику был использован API большой языковой модели ChatGPT. Размер обучающей выборки составил 3672 сообщения. Оценка точности работы словарей и классических алгоритмов обучения проводилась по метрикам и матрицам ошибок, рассмотренным в разделе 2.3. Результаты сравнения моделей на размеченной вручную выборке представлены в таблице 1.

Таблица 1 – Сравнение метрик для словарей kartaslovcent и RuSentiLex

Словарь/модель	Accuracy	Precision	Recall	F1-score
kartaslovcent	0.78	0.79	0.79	0.78
RuSentiLex	0.76	0.76	0.77	0.76
Random Forest	0.67	0.68	0.66	0.61
KNN	0.67	0.66	0.67	0.63

Матрицы ошибок для обоих словарей представлены на рисунке 5.

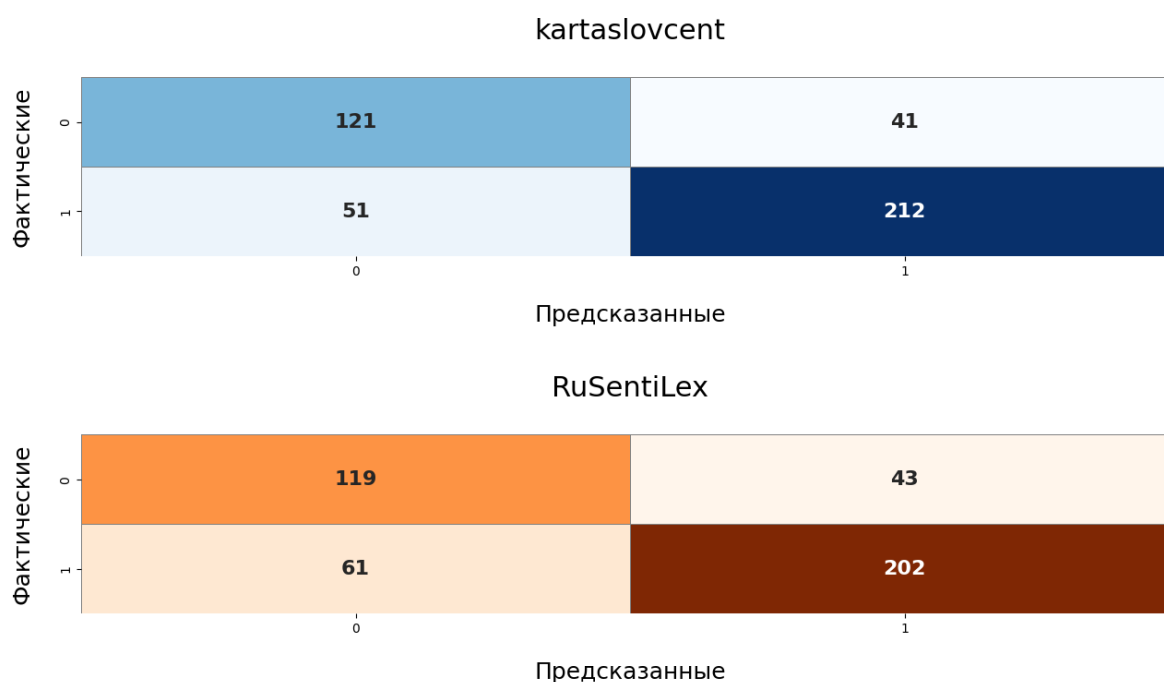


Рисунок 5 – Матрицы ошибок для двух словарей

На основании значения метрик, полученных в ходе проверки точности словарей и классических алгоритмов обучения на тестовой выборке можно сделать вывод о том, что использование словарей позволяет достичь сравнимую с классическими методами точность классификации. Особенная необходимость использования словарных методов для анализа тональности может проявляться в ситуациях отсутствия размеченных текстовых выборок.

Для интеграции в приложение функции тональной оценки была создана вкладка «Анализ среза», позволяющая указать путь к сгенерированному приложением .xlsx файлу, выбрать один или два словаря тональности с помощью которых будет проведен анализ и сгенерировать файл разметки и итоговый файл. Визуализация результатов была произведена с помощью инструментария библиотеки matplotlib [19].

Файл разметки — это промежуточный файл, который в первом столбце содержит исходные тексты, во втором столбце предобработанные тексты, а в третьем столбце находится тональная оценка, полученная применением одного из словарей. Пример фрагмента файла разметки представлен на рисунке 6.

	A	B	C	D
1	text	preprocessed_text	sentiment_dict_1	sentiment_model_2
2	Семья из Башкортостана вошла в число победителей всероссийского конкурса			
	✓ В Министерстве спорта России подвели итоги первого Всероссийского конкурса			
3	— Идея конкурса в том, чтобы позиционировать спорт как образ жизни, который			
	★ Семья Протасовых из Уфы стала серебряным призером всероссийского конкурса	семья башкортостан войт	1	1
4	Особый противопожарный режим в Башкортостане: что нужно знать			
	⚡ С 27 декабря 2024 года по 9 января 2025 года в Башкортостане будет действовать			
5	🚫 Основные запреты в период режима:			
	• Проведение культурно-массовых мероприятий на объектах с нарушениями			
6	• Использование открытого огня (факелов, свечей и т.п.), фейерверков и др.			
	• Запуск пиротехнических изделий вне специально оборудованных мест.			
7	♦ Меры безопасности:			
	• Проведение внеплановых противопожарных инструктажей и тренировок			
8	• Усиление противопожарного контроля на объектах с круглосуточным присутствием			
	• Увеличение численности смен дежурного персонала на таких объектах.			
9	• Организация повышенного контроля за проведением новогодних мероприятий			
	♦ Этот режим — важная мера для обеспечения безопасности и предотвращения			
10	Подробнее ознакомиться с документом можно на официальном интернет-портале	особый противопожарный	1	0

Рисунок 6 – Файл разметки

Итоговый файл — это файл, в котором содержится статистика обработки датасета обоими словарями с указанием процентов позитивных, негативных и нейтральных текстов. После формирования итогового файла генерируется круговая диаграмма, которая выводится в пользовательском интерфейсе и сохраняется в указанную пользователем папку. На рисунке 7 приведен пример такого файла.

1	Метод	Позитивных	Негативных	% Позитивных	% Негативных
2	Kartaslov Dictionary	1300	694	65.20	34.80
3	SentLex Dictionary	1259	735	63.14	36.86

Рисунок 7 – Результаты работы приложения с собранными данными

На рисунке 8 представлено отображение вкладки «Анализ среза».

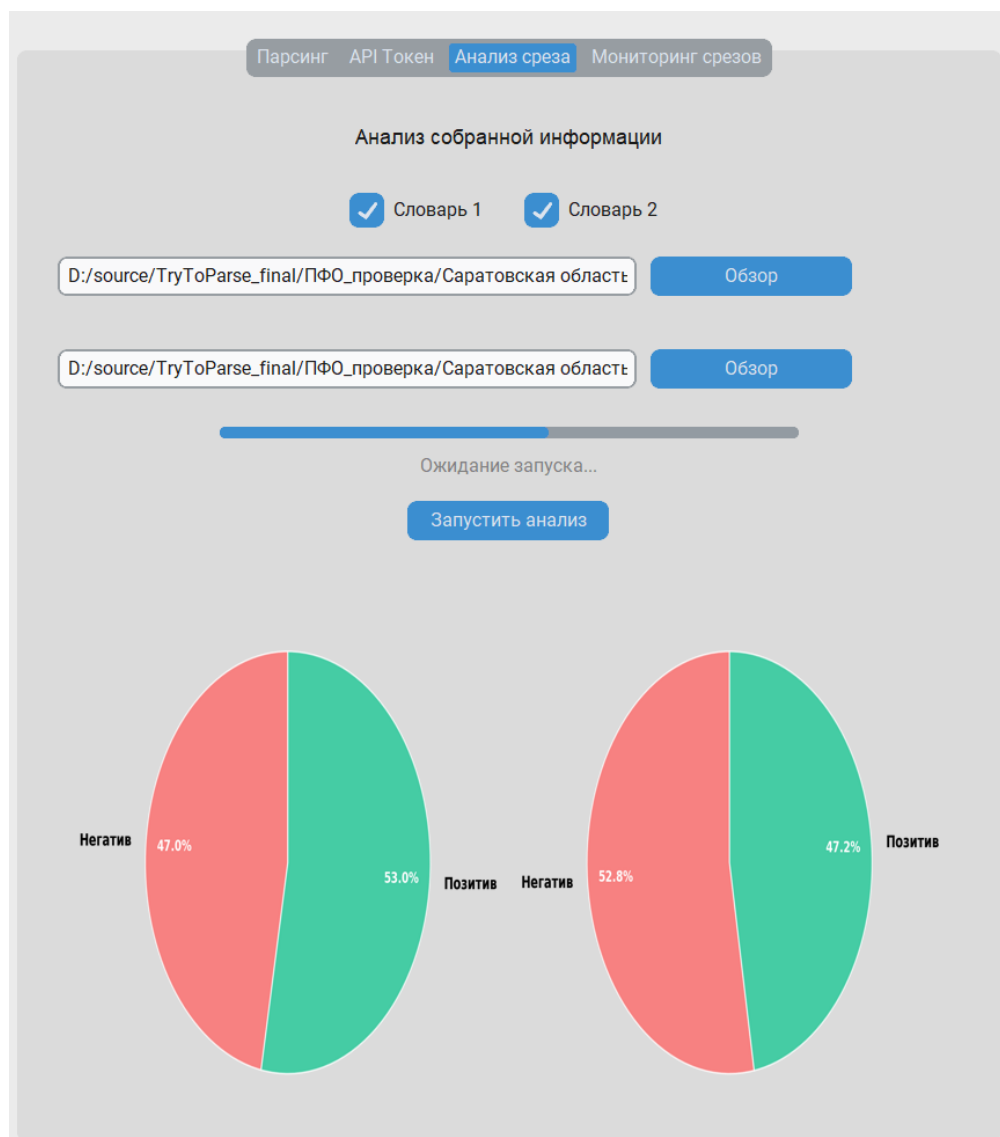


Рисунок 8 – Вкладка «Анализ среза»

Для добавления возможности сбора сводной статистики о результатах работы функции из вкладки «Анализ среза» была создана вкладка «Мониторинг срезов». На данной вкладке пользователю предлагается выбрать каталог, в котором могут содержаться Excel файлы или подкаталоги с Excel файлами, сгенерированными с помощью приложения. Важно, чтобы пользователь сохранял наименования файлов в том виде, в котором их генерирует приложение. На рисунке 9 представлен графический интерфейс вкладки «Мониторинг срезов».

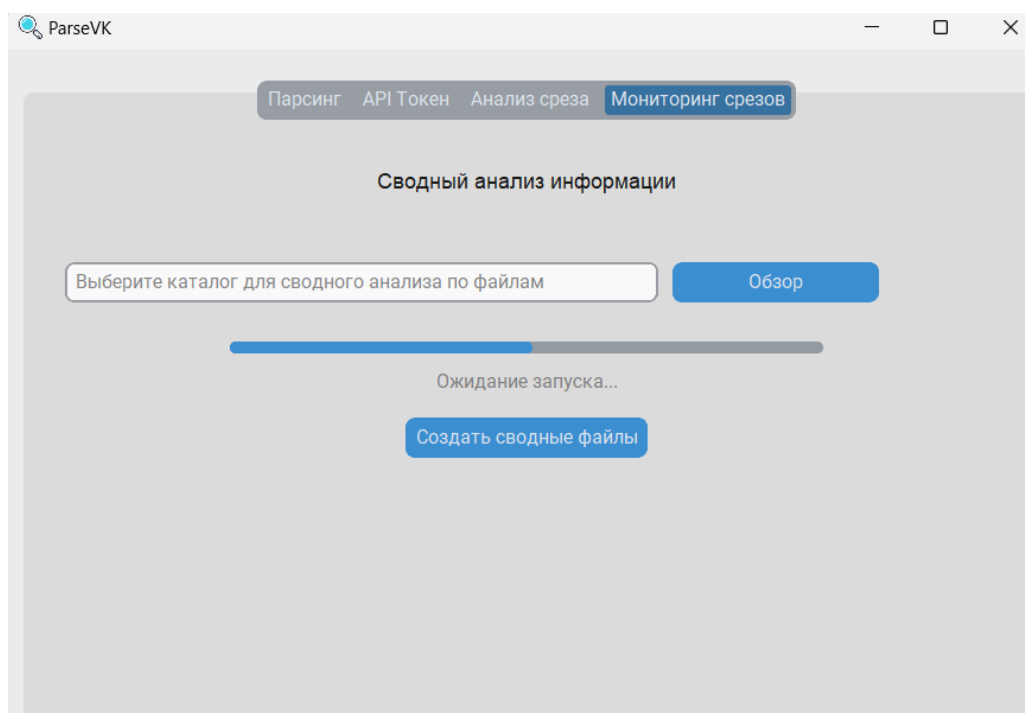


Рисунок 9 – Вкладка «Мониторинг срезов»

После выбора такого каталога и нажатия кнопки запуска сбора сводной статистики в выбранную папку будет сохранен Excel файл, содержащий информацию, что делает удобным дальнейший анализ и визуализацию полученных данных. Кроме того, в выбранной папке сформируется каталог, содержащий графики, характеризующий срезы по количеству собранных текстов в каждый временной период и процент текстов оцененных одним или двумя словарями как положительные. Пример использования описанной функции приведен в разделе 4.

За реализацию сбора сводной информации по Excel файлам отвечает функция `get_summary_analysis_by_folder`. Приведем фрагмент реализации этой функции, в котором собираются данные со всех подкаталогов.

```
# Обход всех папок
for folder in os.listdir(base_folder):
    folder_path = os.path.join(base_folder, folder)
    if os.path.isdir(folder_path): # Проверяем, что это папка
        for file in os.listdir(folder_path):
            if file.endswith("_итоги.xlsx"): # Проверяем, что это Excel
                файл
                file_path = os.path.join(folder_path, file)
            # Извлечение региона и года из названия файла
            temp = file[:-11].split("_")
```

```

region , _ , year , dict_id = temp
year = int(year)
region_and_dict = region+"_"+dict_id
# Чтение данных из Excel файла
df = pd.read_excel(file_path , skiprows=1, nrows=5, header=
    None)
values = df[1].tolist() # Извлекаем значения из второго
    столбца
# Добавляем данные в словарь
if region_and_dict not in data:
data[region_and_dict] = {}
data[region_and_dict][year] = values

```

Результатом работы данной функции является excel-файл, в котором на каждый каталог со срезами один Excel лист с собранными результатами анализа файлов итогов. Пример сводного файла приведен на рисунке 10.

	A	B	C	D	E	F
1		2020	2021	2022	2023	2024
2	Всего текстов	378	1825	2271	2615	1994
3	Всего негативных	227	838	734	1061	694
4	Всего позитивных	151	987	1537	1554	1300
5	Процент негативных	60,05291	45,91781	32,32056	40,57361	34,80441
6	Процент позитивных	39,94709	54,08219	67,67944	59,42639	65,19559
7						
8						
9						

Рисунок 10 – Фрагмент файла сводной статистики

Таким образом, модуль приложения, отвечающий за анализ собранных приложением срезов данных позволяет пользователю осуществить оценку тональности новостных сообщений, выбрать один или два словаря для проведения этой оценки, получить итоговую статистику и ее визуализацию по каждому срезу отдельно или для срезов объединенных в один каталог.

4 Апробация приложения

4.1 Сбор новостей по экономической тематике в региональных СМИ

Рассмотрим применение разработанного парсера для сбора новостных сообщений. Для примера возьмем следующие параметры поиска:

- ключевые слова — «быстро развивающиеся производства; климат для бизнеса, степень демополизации рынков: состояние рынка недвижимости, уровень регионального планирования, деловая инфраструктура; объемы общественных и частных инвестиций, состав и объем муниципальных услуг; инфраструктура, транспорт и связь, степень развития малого предпринимательства»;
- временной период — «01.05.2025–20.05.2025»;
- ключевые слова для поиска сообществ — «Саратовская область»;
- количество сообществ, с которыми поработает парсер, на одну тему — 3;
- режим работы парсера — «Сбор постов»;
- минимальное количество слов в посте или комментарии — 20.

В конечном итоге при заданных параметрах приложение должно сформировать два .xlsx файла с текстами, представляющими собой посты, в которых упоминаются заданные ключевые слова длиной не менее 20 слов, а также вывести количество собранных текстов. По сути собранные продемонстрированным способом данные представляют собой срез данных с новостями выбранного региона за выбранный период с упоминанием конкретных экономических терминов.

В целях применения и апробации созданного приложения была поставлена задача сбора, формирование статистики и анализа тональности годовых срезов экономических новостей по регионам Приволжского федерального округа за последние 5 лет (2020 - 2024). Было принято решение в качестве поискового запроса использовать название региона и осуществлять сбор из 5 новостных сообществ этого региона. Сбор данных осуществлялся с применением фильтрации по минимальному количеству слов в тексте (20) и большому перечню ключевых слов, которые должны присутствовать в отбираемом тексте. Ключевые слова были подобраны в соответствии с целями устойчивого развития ООН [20].

Приведем примененный перечень ключевых слов.

- 1.бедность , распределение доходов , доходы населения , благосостояние , социальная безопасность
- 2.продовольственная безопасность , продовольственный суверенитет , культура питания , регенеративное сельское хозяйство , органическая еда , цена на продукты питания
- 3.психическое здоровье , общественное здравоохранение , психическое благополучие , инвалидность , санитарное просвещение , инфекционные заболевания , детская смертность , планирование семьи , неонатальная смертность , младенческая смертность , здоровье детей , дорожно—транспортные происшествия , репродуктивное здоровье , эпидемии , медицинское страхование
- 4.образование , экологическое образование , профессиональное техническое обучение , бесплатное образование , доступное образование , начальное образование , среднее образование , высшее образование
- 5.гендерное равенство
- 6.рекультивация , эффективность использования воды , истощение грунтовых вод , опустынивание , зеленая инфраструктура
- 7.возобновляемая энергия , ветровая , солнечная , геотермальная , гидроэлектрическая , топливосберегающие технологии , выбросы , парниковый эффект , биотопливо
- 8.занятость , экономический рост , устойчивое развитие , заработная плата , расширение экономических прав и возможностей , малые и средние предприятия , занятость молодежи
- 9.инфраструктура , инвестиции , интернет , промышленная диверсификация
- 10.торговля , финансовый рынок , налогообложение , социальное обеспечение , государственная программа
- 11.общественный транспорт , адаптация к изменению климата , доступное жилье , пешеходная зона , общественные пространства
- 12.природные ресурсы , переработка , промышленная экология , повторное использование , декарбонизация , пищевые отходы
- 13.климат , парниковый газ , глобальное потепление , погода , окружающая среда
- 14.защита водных ресурсов , рыбные запасы
- 15.землепользование , экологическое восстановление земель , сохранение лесов , вырубка лесов , лесовозобновление
- 16.социальная справедливость , правовая система , борьба с терроризмом

Результаты сбора новостей были собраны в структуру, представляющую собой каталог «ПФО», в котором содержится 14 подкаталогов, носящие

названия 14 регионов ПФО. В каждом подкаталоге содержится пять Excel-файлов, содержащих новости конкретного региона за конкретный год (срезы).

На рисунке 11 представлено схематичное изображение части, получившейся структуры.

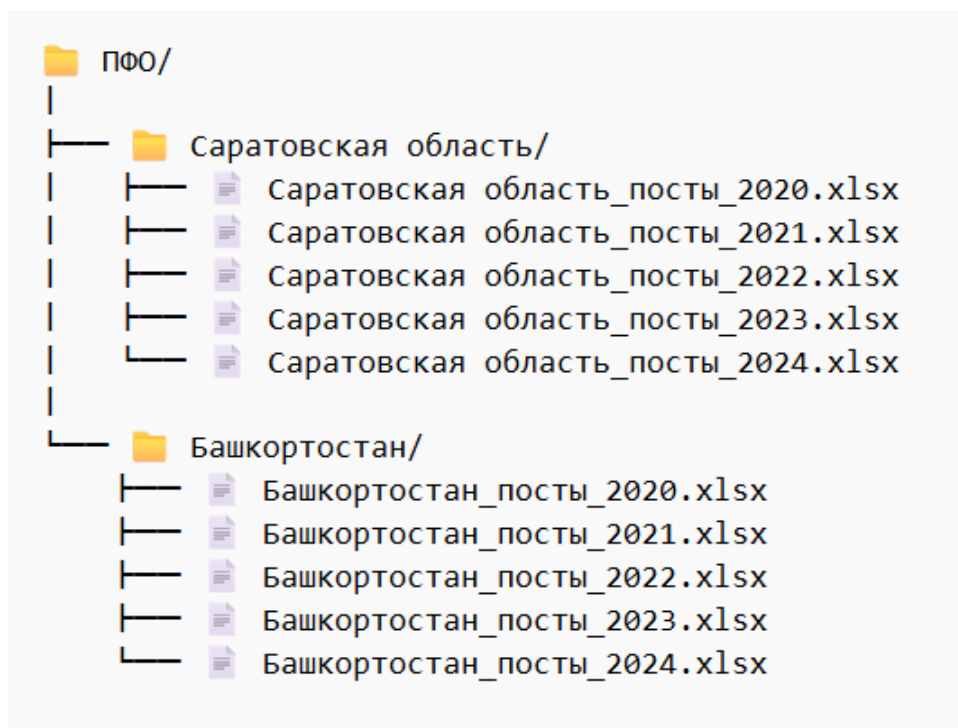


Рисунок 11 – Фрагмент сформированной структуры файлов

Организация файлов в такую структуру и привела к идее создания вкладки «Мониторинг срезов». Далее в работе будет описана апробация приложения в целях анализа экономических тенденций в свете СМИ на примере собранного датасета новостей экономики регионов Приволжского федерального округа.

4.2 Пример применения приложения для анализа экономических новостей регионов ПФО

Для анализа тональности Excel-файлов собранных в структуру, описанную в подразделе 4.1 было принято решение воспользоваться функциональной составляющей вкладки «Анализ среза» следующим образом: проводить анализ с помощью двух словарей, после чего генерируемые файлы разметки, итогов и графики сохранять в ту же папку, в которой храниться анализируемый файл.

На рисунке 12 представлено схематичное изображение структуры файлов после анализа всех срезов.

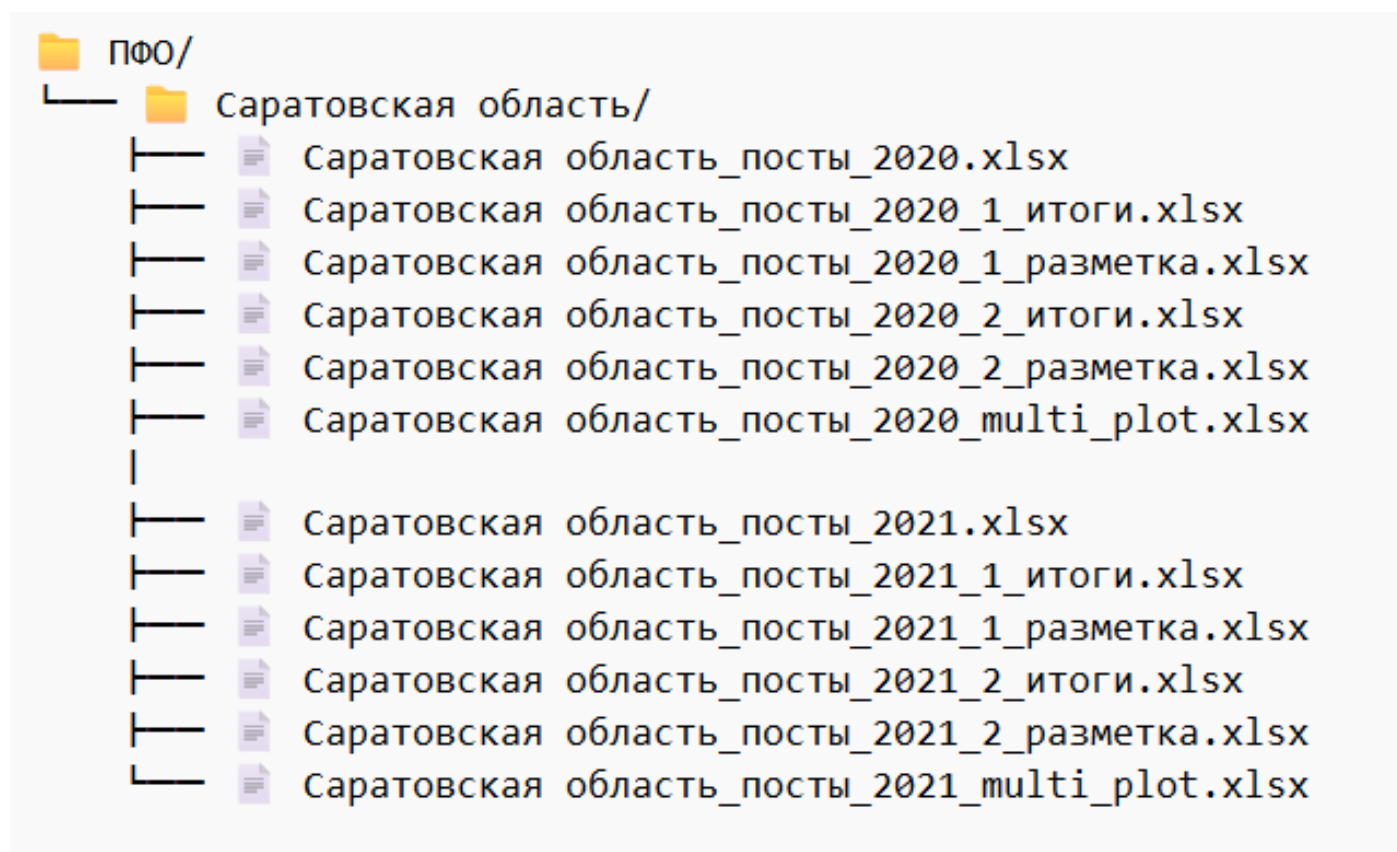
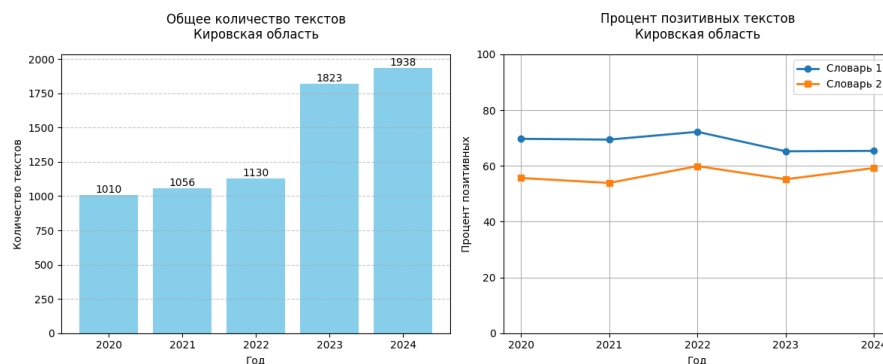
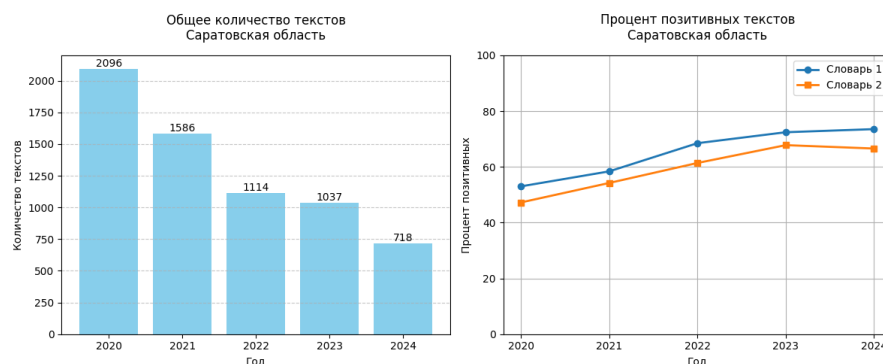


Рисунок 12 – Фрагмент структуры файлов с результатами анализа срезов

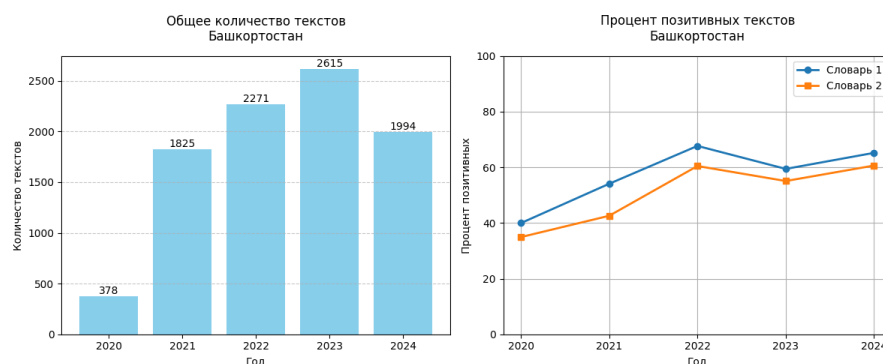
Рассмотрим в качестве примера результаты работы на рисунке 13, на котором представлены столбцовые диаграммы динамики количества экономических новостей по годам(срезам) и диаграммы, отражающие изменения тональности новостных сообщений в трех регионах ПФО в свете процента положительных оценок. Диаграммы были построены с помощью функции вкладки «Мониторинг срезов». чёчё



(а) Динамика Кировской области



(б) Динамика Саратовской области



(с) Динамика Башкортостана

Рисунок 13 – Тренды тональности и количества новостей по годам

В результате анализа графиков построенных для всех 14 регионов ПФО можно отметить некоторые общие тенденции по годовым срезам. Практически во всех регионах год от года растет количество новостей на экономическую тематику. Единственным исключением из этой тенденции является Саратовская область, где наблюдается обратная ситуация. Другой общей тенденцией является рост с 2020 по 2024 год доли новостей, оцениваемых словарями как положительные. Исключением в этом тренде являются Кировская область, где рост доли положительных новостей не наблюдается.

ЗАКЛЮЧЕНИЕ

В бакалаврской работе были проанализированы различные подходы к организации парсинга. По результатам анализа способов веб-скрапинга было принято решение реализовать в приложении модуль, осуществляющий сбор текстовых данных через обращения к API популярной социальной сети Вконтакте.

В приложении был реализован модуль «Парсинг», который позволяет собирать датасеты по различным источникам и тематикам, а также производить отбор текстов по ключевым словам и длине текстовых сообщений.

В бакалаврской работе были рассмотрены различные методы анализа тональности текстов, после чего было принято решение выбрать словарные методы в качестве реализуемых внутри приложения. В качестве словарей, выбранных для использования в приложении были выбраны русскоязычные словари тональности kartaslovcenti RuSentilex. Оценка точности этих словарей была проведена на размеченной вручную выборке с применением метрик оценки точности.

В приложении был реализован модуль, представленный вкладками «Анализ среза» и «Мониторинг срезов». Функциональная составляющая вкладки «Анализ среза» позволяет формировать статистические отчеты и графики, характеризующие тональную оценку текстов, в выбранном пользователем. Во вкладке «Мониторинг срезов» пользователю предоставляется возможность получить сводную статистику по сформированному им каталогу с подкаталогами, которые содержат сгенерированные в разрабатываемом приложении .xlsx файлы. Статистика предоставляется в виде одного итогового отчетного Excel файла по всем подкаталогам и диаграммам и итоговым Excel файлам, которые формируются по данным из каждого подкаталога.

В целях применения и апробации созданного приложения были поставлены задача сбора, анализа тональности и формирования статистики о годовых срезах экономических новостей регионов Приволжского федерального округа за последние 5 лет. Был выполнен сбор данных, в ходе которого к текстам была применена фильтрация по минимальном количеству слов в тексте и большому перечню ключевых слов, которые должны присутствовать в отбираемом тексте. Задачи анализа и сбора статистики получили реализацию в виде графиков.

Дальнейшим направлением развития исследования может стать усовершенствование предобработки собранных данных, процесса подбора и использования ключевых слов для отбора текстовых сообщений. Более точные результаты может дать применение для классификации текстов расширенных словарей и глубокого обучения. Кроме того, развитием исследования может стать интеграция предложенных подходов к анализу новостных сообщений в формирование различных рейтингов региональных субъектов.

Результаты бакалаврской работы были опубликованы в статье Chernyshova, G.; Taran, E.; Firsova, A.; Vavilina, A. Monitoring of Sustainable Development Trends: Text Mining in Regional Media. Sustainability 2025, 17, 3122. <https://doi.org/10.3390/su17073122>.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Документация BeautifulSoup [Электронный ресурс] URL: <https://www.crummy.com/software/BeautifulSoup/bs4/doc.ru/bs4ru.html> (дата обращения: 04.02.2024)
- 2 Web Scraping with Python and BeautifulSoup: A Comprehensive Guide to Data Collection Solutions – USA: O'Reilly, 2015. – 256 с.
- 3 Dimitrios Kouzis-Loukas. Learning Scrapy – USA: Packt Publishing, 2016. – 415 с.
- 4 Ryan M. Web Scraping with Python: Collecting More Data from the Modern Web – USA: O'Reilly, 2018. – 300 с.
- 5 Федеральный закон "Об информации, информационных технологиях и о защите информации" от 27.07.2006 N 149-ФЗ [Электронный ресурс] URL: https://www.consultant.ru/document/cons_doc_LAW_61798/ (дата обращения: 10.03.2025)
- 6 Деревья решений: общие принципы [Электронный ресурс] URL: <https://loginom.ru/blog/decision-tree> (дата обращения: 01.04.2024).
- 7 Сидоров А.В. Алгоритмы создания дерева принятия решений [Электронный ресурс] URL: <https://s.econf.rae.ru/pdf/2014/03/3245.pdf> (дата обращения: 02.04.2024).
- 8 Дерево решений и случайный лес [Электронный ресурс] URL: https://neerc.ifmo.ru/wiki/title=Дерево_решений_случайный_лес (дата обращения: 02.04.2024).
- 9 Метод k-ближайших соседей (K-nearest neighbor) [Электронный ресурс] URL: <https://wiki.loginom.ru/articles/k-nearest-neighbor.html> (дата обращения: 01.04.2024).
- 10 Федорова Е., Рогов О., Демин И. Применение словарей тональности для текстового анализа. — Litres, 2022.
- 11 Метрики в задачах машинного обучения [Электронный ресурс] URL: <https://habr.com/ru/companies/ods/articles/328372/> (дата обращения: 06.01.2025)
- 12 Документация VK API [Электронный ресурс] URL: <https://dev.vk.com/method> (дата обращения: 02.03.2025)

- 13 Работа с файлами XLSX при помощи openpyxl [Электронный ресурс] URL: <https://docs-python.ru/packages/modul-openpyxl/> (дата обращения: 12.10.2024)
- 14 Tkinter – создание графического интерфейса в Python [Электронный ресурс] URL: <https://python-scripts.com/tkinter> (дата обращения: 10.10.2024)
- 15 Документация пакета CustomTkinter [Электронный ресурс] URL: <https://github.com/TomSchimansky/CustomTkinter> (дата обращения: 9.02.2025)
- 16 Лукашевич, Н. В. Создание лексикона оценочных слов русского языка РуСентиЛекс // (OSTIS-2016): материалы VI международной конференции с. 377–382.
- 17 Сайт проекта Kartaslov.ru [Электронный ресурс] URL: <https://kartaslov.ru/> (дата обращения: 11.01.2025)
- 18 Zhang L., Wang S., Liu B. European Chapter of the Association for Computational Linguistics (EACL) [Электронный ресурс] URL: <https://arxiv.org/abs/1704.02813> (дата обращения: 13.01.2025).
- 19 Matplotlib: Visualization with Python [Электронный ресурс] URL: <https://matplotlib.org/> (дата обращения: 01.02.2025)
- 20 Цели в области устойчивого развития [Электронный ресурс] URL: <https://www.un.org/sustainabledevelopment/ru/sustainable-development-goals/> (дата обращения: 04.03.2025)

ПРИЛОЖЕНИЕ А

Программный код реализации графического интерфейса приложения

```
import sys
import customtkinter as ctk
import parsar
import threading
from tkinter import *
import datetime as DT
import time
import os
from PIL import Image
from customtkinter import CTkImage
from tkinter import filedialog, messagebox
import slovari_script_new
from openpyxl import Workbook
import pandas as pd

df_left = []

def resource_path(relative_path):
    """Возвращает корректный путь к ресурсу для работы как в среде
        разработки, так и в собранном приложении."""
    if hasattr(sys, '_MEIPASS'):
        return os.path.join(sys._MEIPASS, relative_path)
    return os.path.join(os.path.abspath("."), relative_path)

def show_graph(graph_path):
    """Отображает график в интерфейсе."""
    global graph_img_label
    image = Image.open(resource_path(graph_path))
    img = CTkImage(light_image=image, size=(600, 400))
    graph_img_label.configure(image=img, text="")
    graph_img_label.image = img

def analyse_file():
    """Анализирует выбранный файл с использованием выбранных
        словарей и выводит результаты."""
```

```

analyse_button.configure(state=DISABLED)
folder_entry.configure(state="readonly")

file_path = file_entry.get()
folder_path = folder_entry.get()

if not file_path.endswith(".xlsx"):
    messagebox.showerror("Ошибка", "Пожалуйста, выберите файл
        формата .xlsx")
    return

file_name = os.path.basename(file_path)

print(f"Обрабатываем файл: {file_name}")

use_first = use_dict1.get()
use_second = use_dict2.get()

if not use_first and not use_second:
    messagebox.showerror("Ошибка", "Выберите хотя бы один
        словарь")
    return

progressbar_analyse.start()
status_label_analyse.configure(text=f"Сбор информации...")

try:
    df = pd.read_excel(file_path)
    if df.empty:
        messagebox.showwarning("Пустой файл", "Файл не содержит
            данных")
        return

    df['text'] = df[df.columns[0]].astype(str)
    df = df[df['text'].str.strip().astype(bool)]

    results = []
    titles = []

    if use_first:
        df1 = df.copy()

```

```

df1["sentiment"] = df1["text"].apply(lambda t:
    slovari_script_new.determine_sentiment_score(t,
    dict_id=1))
results.append(df1)
titles.append("Словарь 1")

output_name_1 = os.path.splitext(file_name)[0]
output_path_1 = os.path.join(folder_path, f"{
    output_name_1}_1_разметка.xlsx")
df1.to_excel(output_path_1, index=False)

total_1 = len(df)
positive_1 = (df1["sentiment"] == 1.0).sum()
negative_1 = (df1["sentiment"] == 0.0).sum()

wb = Workbook()
ws = wb.active
stats = [
    ("Файл", output_name_1),
    ("Всего текстов", total_1),
    ("Негативных", negative_1),
    ("Позитивных", positive_1),
    ("% негативных", negative_1 / total_1 * 100),
    ("% позитивных", positive_1 / total_1 * 100)
]
for row, (label, value) in enumerate(stats, start=1):
    ws[f"A{row}"] = label
    ws[f"B{row}"] = value

stats_path = os.path.join(folder_path, f"{output_name_1}
    _1_итоги.xlsx")
wb.save(stats_path)

if use_second:
    df2 = df.copy()
    df2["sentiment"] = df2["text"].apply(lambda t:
        slovari_script_new.determine_sentiment_score(t,
        dict_id=2))
    results.append(df2)
    titles.append("Словарь 2")

```

```

output_name_2 = os.path.splitext(file_name)[0]
output_path_2 = os.path.join(folder_path, f"{
    output_name_2}_2_разметка.xlsx")
df2.to_excel(output_path_2, index=False)

total_2 = len(df)
positive_2 = (df2["sentiment"] == 1.0).sum()
negative_2 = (df2["sentiment"] == 0.0).sum()

wb = Workbook()
ws = wb.active
stats = [
    ("Файл", output_name_2),
    ("Всего текстов", total_2),
    ("Негативных", negative_2),
    ("Позитивных", positive_2),
    ("% негативных", negative_2 / total_2 * 100),
    ("% позитивных", positive_2 / total_2 * 100)
]
for row, (label, value) in enumerate(stats, start=1):
    ws[f"A{row}"] = label
    ws[f"B{row}"] = value

stats_path = os.path.join(folder_path, f"{output_name_2}
    _2_итоги.xlsx")
wb.save(stats_path)

progressbar_analyse.stop()
analyse_button.configure(state=NORMAL)

status_label_analyse.configure(text=f"Ожидание запуска...")

# Визуализация одной или двух диаграмм
plot_path = slovari_script_new.visualize_multiple(results,
    titles, file_path)
show_graph(plot_path)
folder_entry.configure(state="normal")

except Exception as e:
    messagebox.showerror("Ошибка", f"Не удалось обработать файл
        :\n{e}")

```

```

def get_and_parse_theme(alt_themes, all_keywords, year, date1, date2,
    filename):
    """Основная функция парсинга данных по заданной теме и
        параметрам."""
    if (all_keywords != '') and (date1 != '') and (date2 != '') and
        (alt_themes != '') and (entry_group_count.get() != '') and (
            entry_min_words.get() != '') and not (checkbox_posts ==
                'off' and checkbox_comments == 'off') and (
                entry_token.get() != ''):
        dt1 = DT.datetime.strptime(date1, '%d.%m.%Y').timestamp()
        dt2 = DT.datetime.strptime(date2, '%d.%m.%Y').timestamp()

        status_label.configure(text=f"Ожидание...")
        status_label.pack()
        if date2 < date1:
            status_label.configure(text=f"Введите корректные даты!")
            status_label.pack()

        comments_by_theme = []
        posts_by_theme = []

        btn_start.configure(state=DISABLED)
        entry_keywords_search.configure(state=DISABLED)
        entry_from.configure(state=DISABLED)
        entry_to.configure(state=DISABLED)
        entry_keywords.configure(state=DISABLED)
        entry_group_count.configure(state=DISABLED)
        entry_min_words.configure(state=DISABLED)
        checkbox_posts.configure(state=DISABLED)
        checkbox_comments.configure(state=DISABLED)

        all_words_for_search = alt_themes.split(',')

        theme_part_of_filename = "_".join(all_words_for_search)

        status_label.configure(text="Данные считаны")

        progressbar.start()

```

```

posts_count = 0
comments_count = 0

if checkbox_posts.get() == 0 and checkbox_comments.get() ==
1:
    for word_in_search_keywords in all_words_for_search:
        temp = get_and_parse_comments(
            word_in_search_keywords)
        comments_by_theme.extend(temp)
        comments_count += len(temp)

    status_label.configure(text=f"Печать комментариев")
    time.sleep(2)

    if year == '':
        parsar.file_writer(comments_by_theme, f"{
            theme_part_of_filename}_{date1}_{date2}
            _комментарии", dt1, dt2, filename)
        show_result_only_comments(f"{theme_part_of_filename}
            _{date1}_{date2}_комментарии", len(
                comments_by_theme), status_label)
    else:
        parsar.file_writer(comments_by_theme, f"{
            theme_part_of_filename}_{year}_комментарии", dt1,
            dt2, filename)
        show_result_only_comments(f"{theme_part_of_filename}
            _{year}_комментарии", len(comments_by_theme),
            status_label)

    progressbar.stop()
else:
    if checkbox_posts.get() == 1 and checkbox_comments.get()
    == 0:
        for word_in_search_keywords in all_words_for_search:
            temp = get_and_parse_posts(
                word_in_search_keywords)
            posts_by_theme.extend(temp)
            posts_count += len(temp)

        status_label.configure(text=f"Печать постов")
        time.sleep(2)

```

```

if year == '':
    parsar.file_writer(posts_by_theme, f"{
        theme_part_of_filename}_{date1}_{date2}_посты
        ", dt1, dt2, filename)
else:
    parsar.file_writer(posts_by_theme, f"{
        theme_part_of_filename}_{year}_посты", dt1,
        dt2, filename)
status_label.configure(text=f"Ожидание запуска")
progressbar.stop()
show_result_only_posts(alt_themes, posts_count,
    status_label)
else:
    if checkbox_posts.get() == 1 and checkbox_comments.
        get() == 0:
        for word_in_search_keywords in
            all_words_for_search:
            temp_posts = get_and_parse_posts(
                word_in_search_keywords)
            temp_comments = get_and_parse_comments(
                word_in_search_keywords)

            posts_by_theme.extend(temp_posts)
            comments_by_theme.extend(temp_comments)

            posts_count += len(temp_posts)
            comments_count += len(temp_comments)

        status_label.configure(text=f"Печать постов")
        time.sleep(2)
        if year == '':
            parsar.file_writer(posts_by_theme, f"{
                theme_part_of_filename}_{date1}_{date2}
                _посты", dt1, dt2, filename)
            show_result(f"{theme_part_of_filename}_{
                date1}_{date2}_посты", posts_count,
                comments_count, status_label)
        else:
            parsar.file_writer(posts_by_theme, f"{
                theme_part_of_filename}_{year}_посты",
                dt1, dt2, filename)

```

```

        show_result(f"{theme_part_of_filename}_{year}
                    }_посты", posts_count, comments_count,
                    status_label)

status_label.configure(text=f"Печать
комментариев")
time.sleep(2)
if year == '':
    parsar.file_writer(comments_by_theme, f"{
        theme_part_of_filename}_{date1}_{date2}
        }_комментарии", dt1, dt2, filename)
    show_result(f"{theme_part_of_filename}_{
        date1}_{date2}_комментарии", posts_count,
        comments_count, status_label)
else:
    parsar.file_writer(comments_by_theme, f"{
        theme_part_of_filename}_{year}
        }_комментарии", dt1, dt2, filename)
    show_result(f"{theme_part_of_filename}_{year}
        }_комментарии", posts_count,
        comments_count, status_label)

progressbar.stop()
show_result(alt_themes, posts_count,
            comments_count, status_label)

btn_start.configure(state=NORMAL)
entry_keywords_search.configure(state=NORMAL)
entry_from.configure(state=NORMAL)
entry_to.configure(state=NORMAL)
entry_keywords.configure(state=NORMAL)
entry_group_count.configure(state=NORMAL)
entry_min_words.configure(state=NORMAL)
checkbox_posts.configure(state=NORMAL)
checkbox_comments.configure(state=NORMAL)
else:
    status_label.configure(text=f"Данных недостаточно")
    status_label.pack(pady=5)

```

```

def show_result_only_comments(theme, comments_count, status_label):

```

```

"""Отображает результат парсинга только комментариев."""
status_label.configure(text=f"Файл \n {theme}.xlsx создан\
    \nОтобрано {comments_count} комментариев")
status_label.pack(pady=5)

def show_result(theme, posts_count, comments_count, status_label):
    """Отображает результат парсинга постов и комментариев."""
    status_label.configure(tab_parse,
        text=f"Файл \n {theme}.xlsx создан\nОтобрано\n{
            posts_count} постов\n{comments_count}
            комментариев")
    status_label.pack(pady=5)

def show_result_only_posts(theme, posts_count, status_label):
    """Отображает результат парсинга только постов."""
    status_label.configure(text=f"Файл \n {theme}.xlsx создан\
        \nОтобрано {posts_count} постов")
    status_label.pack(pady=5)

def get_and_parse_posts(one_theme):
    """Парсит посты по заданной теме."""
    dirname = entry_folder_parse.get()
    filename = entry_folder_parse.get()
    date1 = entry_from.get()
    date2 = entry_to.get()
    city = one_theme
    count = entry_group_count.get()
    minimal = entry_min_words.get()
    words = entry_keywords.get()

    if (city != '') and (date1 != '') and (date2 != '') and (
        filename != '') and (count != '') and (minimal != ''):
        dt1 = DT.datetime.strptime(date1, '%d.%m.%Y')
        dt2 = DT.datetime.strptime(date2, '%d.%m.%Y')

        status_label.configure(text=f"Парсинг в сообществах близких
            к теме {one_theme}...")
        posts_by_city = parsar.main_parse_posts(city, dt1.timestamp

```

```

        ( ), dt2.timestamp(), count, filename, minimal, words,
        entry_token.get())
    status_label.configure(text=f"Отобрано {len(posts_by_city)}
        постов")
    time.sleep(2)
    return posts_by_city
else:
    status_label.configure(text=f"Данных недостаточно")
    status_label.pack()
    return []

def get_and_parse_comments(one_theme):
    """Парсит комментарии по заданной теме."""
    filename = entry_folder_parse.get()
    date1 = entry_from.get()
    date2 = entry_to.get()
    theme = one_theme
    count = entry_group_count.get()
    minimal = entry_min_words.get()
    words = entry_keywords.get()

    if (theme != '') and (date1 != '') and (date2 != '') and (
        filename != '') and (count != '') and (minimal != ''):
        dt1 = DT.datetime.strptime(date1, '%d.%m.%Y')
        dt2 = DT.datetime.strptime(date2, '%d.%m.%Y')

        status_label.configure(text=f"Сбор комментов по теме {theme
            }...")
        comments_by_city = parsar.main_parse_comments(theme, dt1.
            timestamp(), dt2.timestamp(), count, filename, minimal,
            words, entry_token.get())
        status_label.configure(text=f"Собрано {len(comments_by_city)
            } комментов")
        time.sleep(2)
        return comments_by_city
    else:
        status_label.configure(text=f"Данных недостаточно")
        status_label.pack()
        return []

```

```

def choose_file():
    """Открывает диалог выбора файла."""
    file_path = filedialog.askopenfilename(
        filetypes=[("Excel files", "*.xlsx"), ("CSV files", "*.csv"),
                    ("All files", ".*")]
    )
    if file_path:
        file_entry.configure(state="normal")
        file_entry.delete(0, "end")
        file_entry.insert(0, file_path)
        file_entry.configure(state="readonly")

def choose_dir():
    """Открывает диалог выбора директории."""
    folder_path = filedialog.askdirectory()
    folder_entry.delete(0, "end")
    folder_entry.insert(0, folder_path)

def choose_dir_parse():
    """Открывает диалог выбора директории для парсинга."""
    folder_path = filedialog.askdirectory()
    entry_folder_parse.insert(0, folder_path)
    entry_folder_parse.configure(state="readonly")

def choose_dir_spec():
    """Открывает диалог выбора директории для сводного анализа."""
    folder_path = filedialog.askdirectory()
    folder_entry_spec.insert(0, folder_path)
    folder_entry_spec.configure(state="readonly")

def analyse_global():
    """Выполняет сводный анализ всех данных в указанной директории
    ."""
    base_folder = folder_entry_spec.get()
    if not os.path.isdir(base_folder):
        messagebox.showerror("Ошибка", "Пожалуйста, выберите

```

```

        существующую папку ")
    return

progressbar_analyse_spec.start()
status_label_analyse_spec.configure(text="Сбор итогов...")

for folder in os.listdir(base_folder):
    folder_path = base_folder + '/' + folder
    slovari_script_new.analyse_combined(folder_path)

status_label_analyse_spec.configure(text="Сбор итогов окончен!")
progressbar_analyse_spec.stop()

# Настройки темы
ctk.set_appearance_mode("light")
ctk.set_default_color_theme("blue")

# Главное окно
root = ctk.CTk()

screen_width = root.winfo_screenwidth()
screen_height = root.winfo_screenheight()

width, height = 700, 800

# Вычисляем позицию для центрирования
x = (screen_width // 2) - (width // 2)
y = (screen_height // 2) - (height // 2)

root.geometry(f"{width}x{height}+{x}+{y}")

root.title("ParseVK")

# Вкладки
tabview = ctk.CTkTabview(master=root, width=680, height=680)
tabview.pack(padx=10, pady=10)
tabview.add("Парсинг")
tabview.add("API Токен")
tabview.add("Анализ среза")
tabview.add("Мониторинг срезов")

```

```

tab_parse = tabview.tab("Парсинг")
tab_token = tabview.tab("API Токен")
tab_stats = tabview.tab("Анализ среза")
tab_stats_spec = tabview.tab("Мониторинг срезов")

# ===== Вкладка "Парсинг" =====

# Период
ctk.CTkLabel(tab_parse, text="Укажите период", font=("Arial", 14)).
    pack(pady=(15, 5))
frame_dates = ctk.CTkFrame(tab_parse, fg_color="transparent")
frame_dates.pack(pady=5)

entry_from = ctk.CTkEntry(frame_dates, placeholder_text="От
    (01.01.2022)", width=200)
entry_to = ctk.CTkEntry(frame_dates, placeholder_text="До
    (01.01.2023)", width=200)

# Метка и поле для года
label_year = ctk.CTkLabel(frame_dates, text="Указать год:")
entry_year = ctk.CTkEntry(frame_dates, placeholder_text="2022",
    width=80)

# Размещаем элементы в сетке
entry_from.grid(row=0, column=1, padx=(0, 20), pady=5, sticky="w")
entry_to.grid(row=0, column=3, padx=(0, 20), pady=5, sticky="w")

label_year.grid(row=0, column=4, padx=(20, 5), pady=5, sticky="e")
entry_year.grid(row=0, column=5, padx=(0, 5), pady=5, sticky="w")

def update_dates_from_year():
    """Обновляет даты начала и конца периода на основе введенного
        года."""
    year = entry_year.get()
    if year.isdigit() and len(year) == 4:
        entry_from.delete(0, "end")
        entry_from.insert(0, f"01.01.{year}")
        entry_to.delete(0, "end")
        entry_to.insert(0, f"01.01.{int(year)+1}")

```

```

# Привязываем обработчик события, когда поле года теряет фокус
entry_year.bind("<FocusOut>", lambda e: update_dates_from_year())

# Выборка
ctk.CTkLabel(tab_parse, text="Выборка", font=("Arial", 14)).pack(
    pady=(10, 5))

entry_keywords_search = ctk.CTkEntry(tab_parse, placeholder_text="
    Ключевые слова для поиска сообществ", width=500)
entry_keywords_search.pack(pady=5)

entry_group_count = ctk.CTkEntry(tab_parse, placeholder_text="
    Количество сообществ", width=500)
entry_group_count.pack(pady=5)

entry_min_words = ctk.CTkEntry(tab_parse, placeholder_text="Мин.
    количество слов", width=500)
entry_min_words.pack(pady=5)

entry_keywords = ctk.CTkEntry(tab_parse, placeholder_text="Ключевые
    слова", width=500)
entry_keywords.pack(pady=5)

# Запись в файл
ctk.CTkLabel(tab_parse, text="Запись в файл", font=("Arial", 14)).
    pack(pady=(10, 5))
# Рамка для выбора папки
frame_folder = ctk.CTkFrame(tab_parse)
frame_folder.pack(pady=10, padx=10, fill="x")

entry_folder_parse = ctk.CTkEntry(frame_folder, width=400,
    placeholder_text="Выберите папку для сохранения итогов")
entry_folder_parse.pack(side="left", padx=(10, 5), pady=10)

folder_button = ctk.CTkButton(frame_folder, text="Обзор", command=
    choose_dir_parse)
folder_button.pack(side="left", padx=5, pady=10)

# Чекбоксы

```

```

frame_checks = ctk.CTkFrame(tab_parse, fg_color="transparent")
frame_checks.pack(pady=5)
var_posts = ctk.BooleanVar(value=False)
var_comments = ctk.BooleanVar(value=False)
checkbox_posts = ctk.CTkCheckBox(frame_checks, text="Посты",
    variable=var_posts)
checkbox_comments = ctk.CTkCheckBox(frame_checks, text="Комментарии",
    variable=var_comments)
checkbox_posts.pack(side="left", padx=10, pady=5)
checkbox_comments.pack(side="left", padx=10, pady=5)

# Прогресс
progressbar = ctk.CTkProgressBar(tab_parse, width=400)
progressbar.set(0.5)
progressbar.pack(pady=10)

# Статус
status_label = ctk.CTkLabel(tab_parse, text="Ожидание запуска...",
    text_color="gray")
status_label.pack()

# Кнопка запуска
btn_start = ctk.CTkButton(tab_parse, text="Запуск",
    command=lambda: threading.Thread(target=
        get_and_parse_theme,
                                args=(
                                    entry_keywords_search.get(),
                                    entry_keywords.get(),
                                    entry_year.get(),
                                    entry_from.get(),
                                    entry_to.get(),
                                    entry_folder_parse.get()),
                                daemon=True).start(),
    font=('Arial', 16, 'bold'),

```

```

        width=70,
        height=30
    )
    btn_start.pack(pady=10)

# ===== Вкладка "Статистика" =====
label_stats = ctk.CTkLabel(tab_stats, text="Анализ собранной
    информации", font=("Arial", 14))
label_stats.pack(pady=20)

# Выбор словарей
dict_select_frame = ctk.CTkFrame(tab_stats)
dict_select_frame.pack(pady=5)

use_dict1 = ctk.BooleanVar(value=True)
use_dict2 = ctk.BooleanVar(value=False)

checkbox_dict1 = ctk.CTkCheckBox(dict_select_frame, text="Словарь
    1", variable=use_dict1)
checkbox_dict2 = ctk.CTkCheckBox(dict_select_frame, text="Словарь
    2", variable=use_dict2)

checkbox_dict1.pack(side="left", padx=10)
checkbox_dict2.pack(side="left", padx=10)

# Рамка для выбора файла
frame_file = ctk.CTkFrame(tab_stats)
frame_file.pack(pady=10, padx=10, fill="x")

file_entry = ctk.CTkEntry(frame_file, width=400, placeholder_text="
    Выберите файл для анализа")
file_entry.pack(side="left", padx=(10, 5), pady=5)

browse_button = ctk.CTkButton(frame_file, text="Обзор", command=
    choose_file)
browse_button.pack(side="left", padx=5, pady=5)

# Рамка для выбора папки
frame_folder = ctk.CTkFrame(tab_stats)
frame_folder.pack(pady=10, padx=10, fill="x")

```

```

folder_entry = ctk.CTkEntry(frame_folder, width=400,
    placeholder_text="Выберите папку для сохранения итогов")
folder_entry.pack(side="left", padx=(10, 5), pady=10)

folder_button = ctk.CTkButton(frame_folder, text="Обзор", command=
    choose_dir)
folder_button.pack(side="left", padx=5, pady=10)

# Прогресс
progressbar_analyse = ctk.CTkProgressBar(tab_stats, width=400)
progressbar_analyse.set(0.5)
progressbar_analyse.pack(pady=5)

# Статус
status_label_analyse = ctk.CTkLabel(tab_stats, text="Ожидание
    запуска...", text_color="gray")
status_label_analyse.pack()

# Кнопка для запуска анализа
analyse_button = ctk.CTkButton(tab_stats, text="Запустить анализ",
    command=lambda: threading.Thread(target=analyse_file, daemon=True)
        .start())
analyse_button.pack(pady=10)

# Место под график
graph_img_label = ctk.CTkLabel(tab_stats, text="[ Здесь появится
    график ]", font=("Arial", 16), text_color="gray")
graph_img_label.pack(pady=20)

# ===== Вкладка "Сводный анализ" =====
label_stats = ctk.CTkLabel(tab_stats_spec, text="Сводный анализ
    информации", font=("Arial", 14))
label_stats.pack(pady=20)

# Рамка для выбора файла
frame_file_spec = ctk.CTkFrame(tab_stats_spec)
frame_file_spec.pack(pady=10, padx=10, fill="x")

folder_entry_spec = ctk.CTkEntry(frame_file_spec, width=400,

```

```

        placeholder_text="Выберите каталог для сводного анализа по файлам
    ")
folder_entry_spec.pack(side="left", padx=(10, 5), pady=10)

browse_button_spec = ctk.CTkButton(frame_file_spec, text="Обзор",
    command=choose_dir_spec)
browse_button_spec.pack(side="left", padx=5, pady=10)

# Прогресс
progressbar_analyse_spec = ctk.CTkProgressBar(tab_stats_spec, width
    =400)
progressbar_analyse_spec.set(0.5)
progressbar_analyse_spec.pack(pady=5)

# Статус
status_label_analyse_spec = ctk.CTkLabel(tab_stats_spec, text="
    Ожидание запуска...", text_color="gray")
status_label_analyse_spec.pack()

# Кнопка для запуска анализа
analyse_button_spec = ctk.CTkButton(tab_stats_spec, text="Создать
    сводные файлы", command=lambda: threading.Thread(target=
    analyse_global, daemon=True).start())
analyse_button_spec.pack(pady=10)

# ===== Вкладка "API Токен" =====

def load_token():
    """Загружает токен из файла при старте приложения."""
    token_file = resource_path('vk_token.txt')
    if os.path.exists(token_file):
        try:
            with open(token_file, 'r') as f:
                token = f.read().strip()
                if token and parsar.check_vk_token(token):
                    entry_token.insert(0, token)
                return True
        except Exception as e:
            print(f"Ошибка при загрузке токена: {e}")
    return False

```

```

def show_help():
    """Показывает справку по работе с токеном."""
    messagebox.showinfo(
        "Справка",
        "Для получения токена вам нужно нажать кнопку 'Получить  

        токен' и следовать следующей инструкции:\n"
        "1. Нажмите кнопку 'Получить токен' для перехода к  

        генератору токенов\n"
        "2. Авторизируйтесь в свою учетную запись ВКонтакте \n"
        "3. Скопируйте полученный токен из адресной строки символы  

        после 'access_token=' до '&expires_in '\n"
        "4. Вставьте токен в поле ввода\n"
        "5. Проверьте токен кнопкой 'Проверить токен'\n"
        "6. Сохраните токен кнопкой 'Сохранить токен'"
    )

def save_token():
    """Сохраняет токен в файл."""
    token = entry_token.get().strip()
    if not token:
        messagebox.showerror("Ошибка", "Поле токена не может быть  

        пустым!")
        return
    try:
        with open(resource_path('vk_token.txt'), 'w') as f:
            f.write(token)
        messagebox.showinfo("Успех", f"Токен успешно сохранен! {  

        resource_path('vk_token.txt')}")
    except Exception as e:
        messagebox.showerror("Ошибка", f"Не удалось сохранить токен:  

        {str(e)}")

def validate_token():
    """Проверяет валидность токена."""
    token = entry_token.get().strip()

    if not token:
        messagebox.showerror("Ошибка", "Поле токена не может быть

```

```

        пустым!")
    return

if not token.startswith('vk1.a.') or len(token) < 50:
    messagebox.showerror("Ошибка", "Токен имеет неверный формат!")
    return

if parsar.check_vk_token(token):
    messagebox.showinfo("Успех", "Токен действителен и работает!")
else:
    messagebox.showerror("Ошибка", "Токен недействителен или не имеет доступа!")

def open_link():
    """Открывает страницу для получения токена VK API."""
    import webbrowser
    webbrowser.open("https://oauth.vk.com/authorize?client_id=51517646&display=page&redirect_uri=https://oauth.vk.com/blank.html&scope=offline&response_type=token&v=5.131")

label_token = ctk.CTkLabel(tab_token, text="Токен VK API", font=("Arial", 14))
label_token.pack(pady=5)
entry_token = ctk.CTkEntry(tab_token, width=500, placeholder_text="Введите ваш токен")
entry_token.pack(pady=5)

# Загружаем токен при инициализации
if load_token():
    messagebox.showinfo("Результат загрузки токена", "Валидный токен для работы уже загружен из сохранения")
else:
    messagebox.showinfo("Результат загрузки токена", "Токен для работы не был загружен из сохранения или не валиден.\n"
                                                                "Для получения, проверки и сохранения")

```

токена
перейдите на

соответствующую
вкладку")

```
# Фрейм для кнопок сохранения и проверки
button_top_frame = ctk.CTkFrame(tab_token)
button_top_frame.pack(pady=10)

# Кнопка проверки
validate_button = ctk.CTkButton(
    button_top_frame,
    text="Проверить токен",
    command=validate_token,
    fg_color="#2aa9ff"
)
validate_button.pack(side="left", padx=5)

# Кнопка сохранения
save_button = ctk.CTkButton(
    button_top_frame,
    text="Сохранить токен",
    command=save_token
)
save_button.pack(side="left", padx=5)

# Фрейм для нижних кнопок
button_bottom_frame = ctk.CTkFrame(tab_token)
button_bottom_frame.pack(padx=10, pady=10)

link_button = ctk.CTkButton(
    button_bottom_frame,
    text="Получить токен",
    command=open_link,
    fg_color="transparent",
    border_width=1,
    text_color=("gray10", "gray90")
)
```

```
link_button.pack(side="left", padx=5)
```

```
help_button = ctk.CTkButton(  
    button_bottom_frame,  
    text="Помощь",  
    width=30,  
    command=show_help,  
    fg_color="transparent",  
    border_width=1,  
    text_color=("gray10", "gray90")  
)
```

```
help_button.pack(side="right", padx=5)
```

```
# Запуск
```

```
root.iconbitmap(resource_path('favicon.ico'))
```

```
root.mainloop()
```

ПРИЛОЖЕНИЕ Б

Программный код реализации парсера

```
# coding: utf8
import os
import sys
import requests
import time
import re
from nltk.stem import SnowballStemmer
import openpyxl

snowball = SnowballStemmer(language="russian")

def resource_path(relative_path):
    """Возвращает корректный путь к ресурсу для работы как в среде
    разработки, так и в собранном приложении."""
    if hasattr(sys, '_MEIPASS'):
        return os.path.join(sys._MEIPASS, relative_path)
    return os.path.join(os.path.abspath("."), relative_path)

def get_comments(domain, dt1, dt2, minimal, words, user_token):
    """Получает комментарии из постов сообщества с учетом фильтров
    ."""
    version = "5.131"
    searched_comments = []
    all_comments = []

    searched_posts = get_posts(domain, dt1, dt2, words)
    start = time.time()
    offset = 0

    for post in searched_posts:
        offset1 = 0
        while offset1 < post['comments']['count']:
            print(offset)
            time.sleep(0.5)
            code2 = """return API.wall.getComments({"owner_id": ""
                + f"{post['owner_id']}" + f""", "post_id": {post['id
                ']}"" + """, "v": \"5.131\\\""" + f""", "count": 100"""
```

```

+ """});"""

response2 = requests.post(
    "https://api.vk.com/method/execute",
    data={
        "code": code2,
        "access_token": user_token,
        "v": version
    }
)
comments = response2.json()[ 'response '][ 'items ']

for comment in comments:
    if len(comment[ 'text '].split()) >= int(minimal):
        searched_comments.append(comment)

all_comments.extend(searched_comments)
offset1 += 100
searched_comments.clear()
offset += post[ 'comments '][ 'count ']

end = time.time() - start
print(f"Обработано {offset} комментариев за {end} секунд из них {
    len(all_comments)} отобрано")
return all_comments

def get_posts(domain, dt1, dt2, words):
    """Получает посты из сообщества с учетом фильтров по ключевым
        словам."""
    token = "
d8e76f9bd8e76f9bd8e76f9b21dbf57755dd8e7d8e76f9bbb30f36f666b129e1c0b2
"
    version = "5.131"
    searched_posts = []
    all_searched_posts = []

    url = f'https://api.vk.com/method/wall.get?access_token=
d8e76f9bd8e76f9bd8e76f9b21dbf57755dd8e7d8e76f9bbb30f36f666b129e1c0b2
&v=5.131&domain={domain}'

```

```

try:
    posts_response = requests.get(url)
except:
    print("Сообщество пропущено")
else:
    posts_count = posts_response.json()['response'][ 'count' ]
    print(f"Всего постов в группе:{ posts_count }")

    offset = 0
    start = time.time()
    regular_expression = ''

    keywords = words.lower().split(',')

    if len(keywords[0].split()) > 1:
        for keyword in keywords[0].split():
            word = snowball.stem(keyword)
            regular_expression += word + '.*'
        regular_expression += '|'

        for keyword in reversed(keywords[0].split()):
            word = snowball.stem(keyword)
            regular_expression += word + '.*'
    else:
        regular_expression += snowball.stem(keywords[0])

    if len(keywords) > 1:
        for i in range(1, len(keywords)):
            regular_expression += '|'

            for keyword in keywords[i].split():
                word = snowball.stem(keyword)
                regular_expression += word + '.*'

            regular_expression += '|'

            for keyword in keywords[i].split():
                word = snowball.stem(keyword)
                regular_expression += word + '.*'

    while offset < posts_count:

```

```

code1 = """return API.wall.get({"domain": "" + f"\{
    domain}\\" + "", "offset": "" + str(
        offset) + "", "count": 100, "extended": 0, "v":
        "5.131"});"""

response1 = requests.post(
    "https://api.vk.com/method/execute",
    data={
        "code": code1,
        "access_token": token,
        "v": version
    }
)
data1 = response1.json()[ 'response '][ 'items ' ]

if data1[50][ 'date ' ] < dt1:
    break

for post in data1:
    if (re.search(rf"{regular_expression}", rf"{post[ '
        text ' ].lower()}" ) is not None) and (post[ 'date ' ]
        < dt2) and (post[ 'date ' ] > dt1):
        searched_posts.append(post)

offset += 100
end = time.time() - start

all_searched_posts.extend(searched_posts)
print(f"Обработано {offset} постов за {end} секунд из
    них {len(all_searched_posts)} отобрано")
searched_posts.clear()
finally:
    return all_searched_posts

def file_writer(data, theme, dt1, dt2, output_path):
    """Записывает данные в Excel файл с фильтрацией по дате."""
    workbook = openpyxl.Workbook()
    worksheet = workbook.active
    i = 1

```

```

for post in data:
    if (post['date'] < dt2) and (post['date'] > dt1) and (post['text'] != ' '):
        worksheet['A' + str(i)] = post['text']
        i += 1

filename = theme + ".xlsx"
output_path_final = os.path.join(output_path, filename)
workbook.save(output_path_final)

def parse_groups_posts(city, groups, dt1, dt2, region, minimal, words):
    """Парсит посты из списка групп."""
    posts = []
    for group in groups:
        posts += get_posts(group['screen_name'], dt1, dt2, words)
    return posts

def parse_groups_comments(groups, dt1, dt2, minimal, words, user_token):
    """Парсит комментарии из списка групп."""
    comments = []
    for group in groups:
        print(f"Группа {group['name']}")
        comments += get_comments(group['screen_name'], dt1, dt2, minimal, words, user_token)
    return comments

def main_parse_posts(city, dt1, dt2, count, region, minimal, words, user_token):
    """Основная функция для парсинга постов по городу."""
    version = "5.131"

    response = requests.get(
        "https://api.vk.com/method/groups.search",
        params={
            "access_token": user_token,
            "v": version,

```

```

        "q": f"{city}",
        "type": "page",
        "sort": 0,
        "count": int(count),
    }
)
print("here")

groups = response.json()[ 'response '][ 'items ']
groups_posts = parse_groups_posts(city, groups, dt1, dt2, region
    , minimal, words)

return groups_posts

def main_parse_comments(city, dt1, dt2, count, filename, minimal,
    words, user_token):
    """Основная функция для парсинга комментариев по городу."""
    version = "5.131"

    response = requests.get(
        "https://api.vk.com/method/groups.search",
        params={
            "access_token": user_token,
            "v": version,
            "q": f"{city}",
            "type": "page",
            "sort": 1,
            "count": int(count),
        }
    )
    groups = response.json()[ 'response '][ 'items ']
    groups_comments = parse_groups_comments(groups, dt1, dt2,
        minimal, words, user_token)

    return groups_comments

def check_vk_token(token):
    """Проверяет валидность токена VK API."""
    try:

```

```

response = requests.get(
    'https://api.vk.com/method/users.get',
    params={'access_token': token, 'v': '5.131'})
data = response.json()

return response.status_code == 200 and 'error' not in data
except Exception as e:
    print(f"Ошибка при проверке токена: {e}")
    return False

```

ПРИЛОЖЕНИЕ В

Программный код реализации анализа тональности

```
# coding: utf-8

import pandas as pd
import os
import sys
import logging
from openpyxl import Workbook
import pymorphy3
import matplotlib.pyplot as plt
from nltk.corpus import stopwords

def resource_path(relative_path):
    """Возвращает абсолютный путь к ресурсу (работает и из .py, и из
    .exe)."""
    if hasattr(sys, '_MEIPASS'):
        return os.path.join(sys._MEIPASS, relative_path)
    return os.path.join(os.path.abspath("."), relative_path)

# Настройка логов
logging.basicConfig(
    filename='sentiment_analysis.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

morph = pymorphy3.MorphAnalyzer()
russian_stopwords = set(stopwords.words("russian"))
df_left = []

# Можно указать относительный путь к папке данных:
base_folder = resource_path("data")

def load_sentiment_dict(csv_path, dict_id):
    """Загружает словарь тональностей из CSV файла."""
    try:
        if dict_id == 1:
```

```

        sentiment_data = pd.read_csv(csv_path, usecols=['term',
            'value'], sep=";")
        return dict(zip(sentiment_data["term"], sentiment_data["
            value"]))
    else:
        sentiment_data_1 = pd.read_csv(csv_path, usecols=['term
            ', 'value'], sep=",")
        return dict(zip(sentiment_data_1["term"],
            sentiment_data_1["value"]))
except Exception as e:
    logging.error(f"Ошибка загрузки словаря: {e}")
    raise


def determine_sentiment_score(text, dict_id):
    """Определяет тональность текста на основе выбранного словаря
    ."""
    if dict_id == 1:
        csv_name = "kartaslovsent.csv"
    else:
        csv_name = "rusentilex.csv"
    sentiment_csv_path = resource_path(csv_name)
    sentiment_dict = load_sentiment_dict(sentiment_csv_path, dict_id
    )

    if not isinstance(text, str) or not text.strip():
        return 0.0

    words = text.split()
    score = 0
    count = 0

    for word in words:
        try:
            normal_form = morph.parse(word.lower())[0].normal_form
            if normal_form in sentiment_dict and normal_form not in
                russian_stopwords:
                score += sentiment_dict[normal_form]
                count += 1
        except:
            continue

```

```

    if count == 0:
        df_left.append(text)
        return 0.0

    average_score = score / count
    return 1.0 if average_score >= 0.2 else 0.0

def visualize_sentiment(df, file_name):
    """Создает круговую диаграмму распределения тональности и
        сохраняет её."""
    sentiment_counts = df["sentiment"].value_counts()
    labels = ['Негатив', 'Позитив']
    sizes = [
        sentiment_counts.get(0.0, 0),
        sentiment_counts.get(1.0, 0)
    ]

    plt.figure(figsize=(8, 6))
    plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90,
            colors=['#ff9999', '#99ff99'])
    plt.title(f'Распределение тональности ({file_name})')
    plt.axis('equal')

    plot_path = os.path.join(
        os.path.dirname(file_name),
        f"{os.path.splitext(os.path.basename(file_name))[0]}_plot.
        png"
    )
    plt.savefig(plot_path)
    plt.close()
    logging.info(f"График сохранен: {plot_path}")

    return plot_path

def visualize_multiple(dfs, titles, base_file_path):
    """Создает несколько круговых диаграмм для сравнения результатов
        ."""
    fig, axes = plt.subplots(1, len(dfs), figsize=(6 * len(dfs), 6))

```

```

if len(dfs) == 1:
    axes = [axes]

for ax, df, title in zip(axes, dfs, titles):
    sentiment_counts = df["sentiment"].value_counts()
    sizes = [sentiment_counts.get(0.0, 0), sentiment_counts.get(
        1.0, 0)]
    labels = ['Негатив', 'Позитив']

    ax.pie(sizes, labels=labels, autopct='%1.1f%%', startangle
        =90, colors=['#ff9999', '#99ff99'])
    ax.set_title(f'{title}')

plt.tight_layout()

plot_path = os.path.join(
    os.path.dirname(base_file_path),
    f"{os.path.splitext(os.path.basename(base_file_path))[0]}
        _multi_plot.png"
)
plt.savefig(plot_path)
plt.close()
return plot_path

def analyse_combined(base_folder_analyse):
    """Анализирует все файлы в указанной папке и сохраняет
        результаты."""
    for file_name in os.listdir(base_folder_analyse):
        folder_path = os.path.join(base_folder_analyse, file_name)

        for file_name in os.listdir(base_folder_analyse):
            if not (file_name.endswith('посты.xlsx') or file_name.
                endswith('комментарии.xlsx')):
                continue

            print(f"Обрабатываем файл: {file_name}")
            logging.info(f"Обработка файла: {file_name}")
            file_path = os.path.join(folder_path, file_name)

```

```

try:
    df = pd.read_excel(file_path)
    if df.empty:
        logging.warning(f"Файл пуст: {file_name}")
        continue

    text_column = df.columns[0]
    df = df.rename(columns={text_column: 'text'})
    df = df.dropna(subset=['text'])
    df['text'] = df['text'].astype(str)
    df = df[df['text'].str.strip().astype(bool)]

    df["sentiment"] = df["text"].apply(lambda t:
        determine_sentiment_score(t, dict_id=1))

    if df_left:
        df = df[~df["text"].isin(df_left)]

    output_name = os.path.splitext(file_name)[0]
    output_path = os.path.join(folder_path, f"{
        output_name}_разметка.xlsx")
    df.to_excel(output_path, index=False)
    logging.info(f"Результаты сохранены: {output_path}")

    visualize_sentiment(df, file_path)

    total = len(df)
    positive = (df["sentiment"] == 1.0).sum()
    negative = (df["sentiment"] == 0.0).sum()

    wb = Workbook()
    ws = wb.active
    stats = [
        ("Файл", output_name),
        ("Всего текстов", total),
        ("Негативных", negative),
        ("Позитивных", positive),
        ("% негативных", negative / total * 100),
        ("% позитивных", positive / total * 100)
    ]

```

```

        for row, (label, value) in enumerate(stats, start=1)
            :
            ws[f"A{row}"] = label
            ws[f"B{row}"] = value

        stats_path = os.path.join(folder_path, f"{
            output_name}_итоги.xlsx")
        wb.save(stats_path)
        logging.info(f"Статистика сохранена: {stats_path}")

    except Exception as e:
        logging.error(f"Ошибка обработки файла {file_name}:
            {e}")
        continue

    logging.info("Обработка завершена!")
    print("Анализ завершен. Проверьте папки с результатами.")

# Пример вызова функции
analyse_combined("D:\\source\\TryToParse_2025\\ПФО_проверка")

```